Microservicios con Spring

© JMA 2020. All rights reserved

INTRODUCCIÓN

© JMA 2020. All rights reserved

Antecedentes

- El estilo de microservicio surge como alternativa al estilo monolítico.
- Una aplicación monolítica esta construida como una sola unidad. Las aplicaciones empresariales a menudo están integradas en tres partes principales:
  - una interfaz de usuario del lado del cliente (que consta de páginas HTML y javascript que se ejecutan en un navegador en la máquina del usuario)
  - una base de datos (que consta de muchas tablas insertadas en una instancia de bases de datos común y generalmente relacional)
  - una aplicación del lado del servidor que manejará las solicitudes HTTP, ejecutará la lógica del dominio, recuperará y actualizará los datos de la base de datos, y seleccionará y completará las vistas HTML que se enviarán al navegador.
- Esta aplicación del lado del servidor es un monolito, un único ejecutable lógico. Cualquier cambio en el sistema implica crear e implementar una nueva versión de la aplicación del lado del servidor.
- Cuando las aplicaciones escalan y se vuelven muy grandes, una aplicación monolítica construida como una sola unidad presenta serios problemas.

© JMA 2020. All rights reserved

SOA

- El concepto de dividir una aplicación en partes discretas no es nuevo. La idea para microservicios se origina en el patrón SOA de diseño de arquitectura orientado a servicios más amplio. La arquitectura orientada a servicios (SOA) es un método de desarrollo de software que utiliza componentes de software llamados servicios para crear aplicaciones empresariales. Cada uno de estos servicios brinda una capacidad empresarial y, además, pueden comunicarse también con el resto de servicios mediante diferentes plataformas y lenguajes. Los desarrolladores usan SOA para reutilizar servicios en diferentes sistemas o combinar varios servicios independientes para realizar tareas complejas.
- Sin embargo, a diferencia de la arquitectura orientada a servicios, una arquitectura de microservicios (como su nombre indica) debe contener servicios que son explícitamente pequeños y ligeros y que son desplegables de forma independiente. Los objetivos son:
  - Poder utilizar diferentes tecnologías por cada servicio (Java EE, Node, …)
  - Permitir que cada servicio tenga un ciclo de vida independiente, es decir, versión independiente del resto, inclusive equipos de desarrollo diferentes.
  - Al ser servicios sin dependencia entre sí (especialmente de sesión), poder ejecutar el mismo en varios puertos, colocando un balanceador delante.
  - Poder crear instancias en servidores de diferentes regiones, lo que permitirá crecer (tanto verticalmente como horizontal) sin necesidad de cambiar el código fuente.

© JMA 2020. All rights reserved

La nube

- La nube está cambiando la forma en que se diseñan y protegen las aplicaciones. En lugar de ser monolitos, las aplicaciones se descomponen en servicios menores y descentralizados. Estos servicios se comunican a través de APIs, mediante el uso de eventos o de mensajería asincrónica. Las aplicaciones se escalan horizontalmente, agregando nuevas instancias, tal y como exigen las necesidades.
- Estas tendencias agregan nuevos desafíos:
  - El estado de las aplicaciones se distribuye.
  - Las operaciones se realizan en paralelo y de forma asincrónica.
  - Las aplicaciones deben ser resistentes cuando se produzcan errores.
  - Las aplicaciones son continuamente atacadas por actores malintencionados.
  - Las implementaciones deben estar automatizadas y ser predecibles.
  - La supervisión y la telemetría son fundamentales para obtener una visión general del sistema.

© JMA 2020. All rights reserved

Cambio de paradigma

Local tradicional

- Monolítica
- Diseñada para una escalabilidad predecible
- Base de datos relacional
- Procesamiento síncrono
- Diseño para evitar errores (MTBF)
- Actualizaciones grandes, ocasionales
- Administración manual
- Servidores en copo de nieve

Nube moderna

- Descompuesto
- Diseñado para un escalado elástico
- Persistencia poliglota (combinación de tecnologías de almacenamiento)
- Procesamiento asíncrono
- Diseño resiliente a errores (MTTR)
- Pequeñas actualizaciones, frecuentes
- Administración automatizada
- Infraestructura inmutable

© JMA 2020. All rights reserved

Estilos de arquitectura

- **N Niveles**: es la arquitectura tradicional para aplicaciones empresariales.
- **Web-queue-worker**: solución puramente PaaS, la aplicación tiene un front-end web que controla las solicitudes HTTP y un trabajador back-end que realiza tareas de uso intensivo de la CPU u operaciones de larga duración. El front-end se comunica con el trabajador a través de una cola de mensajes asincrónicos.
- **Orientada a servicios (SOA)**: término sobre utilizado pero, como denominador común, significa que se estructura descomponiéndola en varios servicios que se pueden clasificar en tipos diferentes, como subsistemas o niveles.
- **Microservicios**: en un sistema que requiere alta escalabilidad y alto rendimiento, la arquitectura de microservicios se descompone en muchos servicios pequeños e independientes.
- **Basadas en eventos**: usa un modelo de publicación-suscripción (pub-sub), en el que los productores publican eventos y los consumidores de suscriben a ellos. Los productores son independientes de los consumidores y estos, a su vez, son independientes entre sí.
- **Big Data**: permite dividir un conjunto de datos muy grande en fragmentos, realizando un procesamiento paralelo en todo el conjunto, con fines de análisis y creación de informes.
- **Big compute**: también denominada informática de alto rendimiento (HPC), realiza cálculos en paralelo en un gran número (miles) de núcleos.

© JMA 2020. All rights reserved

Arquitectura de microservicios

- El término "Microservice Architecture" ha surgido en los últimos años (2011) para describir una forma particular de diseñar aplicaciones de software como conjuntos de servicios de implementación independiente.
- Si bien no existe una definición precisa de este estilo arquitectónico, existen ciertas características comunes en torno a la organización en torno a la capacidad empresarial, la implementación automatizada, la inteligencia en los puntos finales y el control descentralizado de lenguajes y datos. (Martin Fowler)
- El estilo arquitectónico de microservicio es un enfoque para desarrollar una aplicación única como un conjunto de pequeños servicios, cada uno ejecutándose en su propio proceso y comunicándose con mecanismos ligeros, a menudo una API de recursos HTTP.
- Estos servicios se basan en capacidades empresariales y se pueden desplegar de forma independiente mediante mecanismos de implementación totalmente automatizada.
- Hay un mínimo de administración centralizada de estos servicios, que puede escribirse en diferentes lenguajes de programación y usar diferentes tecnologías de almacenamiento de datos.

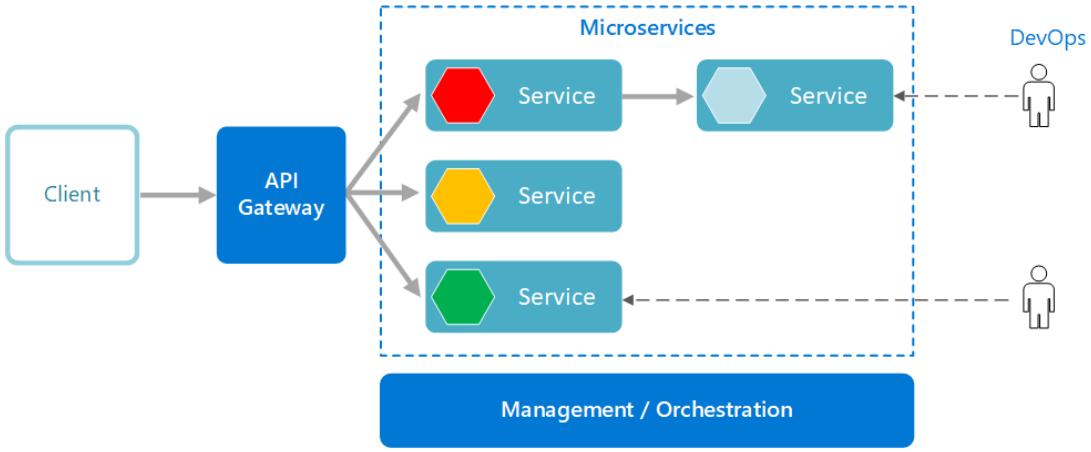
© JMA 2020. All rights reserved

Arquitectura de microservicios

- Una arquitectura de microservicios consta de una colección de servicios autónomos y pequeños. Los servicios son independientes entre sí y cada uno debe implementar una funcionalidad de negocio individual.
- Los microservicios son pequeños e independientes, y están acoplados de forma imprecisa. Un único equipo reducido de programadores puede escribir y mantener un servicio.
- Cada servicio es un código base independiente, que puede administrarse por un equipo de desarrollo pequeño.
- Los servicios pueden implementarse de manera independiente. Un equipo puede actualizar un servicio existente sin tener que volver a generar e implementar toda la aplicación.
- Los servicios son los responsables de conservar sus propios datos o estado externo. Esto difiere del modelo tradicional, donde una capa de datos independiente controla la persistencia de los datos.
- Los servicios se comunican entre sí mediante APIs bien definidas. Los detalles de la implementación interna de cada servicio se ocultan frente a otros servicios.
- No es necesario que los servicios compartan la misma pila de tecnología, las bibliotecas o los marcos de trabajo.
- El estilo de microservicio surge como alternativa al estilo monolítico.

© JMA 2020. All rights reserved

Arquitectura de microservicios



© JMA 2020. All rights reserved

Arquitectura de microservicios

- Además de los propios servicios, hay otros componentes que aparecen en una arquitectura típica de microservicios:
  - **Administración e implementación**: Este componente es el responsable de la colocación de servicios en los nodos, la identificación de errores, el reequilibrio de servicios entre nodos, etc. Normalmente, este componente es una tecnología estándar, como Kubernetes, en lugar de algo creado de forma personalizada.
  - **Puerta de enlace de API**: Es el punto de entrada para los clientes. En lugar de llamar a los servicios directamente, los clientes llaman a la puerta de enlace de API, que reenvía la llamada a los servicios apropiados en el back-end. Entre las ventajas de usar una puerta de enlace de API se encuentran las siguientes:
    - Desacoplan los clientes de los servicios. Los servicios pueden cambiar de versión o refactorizarse sin necesidad de actualizar todos los clientes.
    - Los servicios pueden utilizar los protocolos de mensajería que no son fáciles de usar para un servicio web, como AMQP.
    - La puerta de enlace de API puede realizar otras funciones transversales como la autenticación, el registro, la terminación SSL y el equilibrio de carga.

© JMA 2020. All rights reserved

Ventajas de los microservicios

- **Agilidad**: Dado que microservicios se implementan de forma independiente, resulta más fácil de administrar las correcciones de errores y las versiones de características. Puede actualizar un servicio sin volver a implementar toda la aplicación y revertir una actualización si algo va mal.
- **Equipos pequeños y centrados**: Un microservicio debe ser lo suficientemente pequeño como para que un solo equipo de características lo pueda compilar, probar e implementar. Los equipos pequeños favorecen la agilidad.
- **Base de código pequeña**: En las aplicaciones monolíticas, con el paso del tiempo se da la tendencia de que las dependencias del código se acaben enredarse, por lo que para agregar una nueva característica, es preciso tocar el código en muchos lugares. Al no compartir el código ni los almacenes de datos, la arquitectura de microservicios minimiza las dependencias y resulta más fácil agregar nuevas características.
- **Mezcla de tecnologías**: Los equipos pueden elegir la tecnología que mejor se adapte al servicio de una combinación de pilas de tecnología, según corresponda.
- **Aislamiento de defectos mejorado**: Si un microservicio individual no está disponible o genera problemas, no interrumpe toda la aplicación, siempre que los microservicios de nivel superior estén diseñados para controlar los errores correctamente (por ejemplo, circuit break).
- **Escalabilidad**: Los servicios se pueden escalar de forma independiente, lo que permite escalar horizontalmente los subsistemas que requieren más recursos, sin tener que escalar horizontalmente toda la aplicación. Mediante un orquestador como Kubernetes o Service Fabric se puede empaquetar una mayor densidad de servicios en un solo host, lo que aumenta la eficacia en el uso de los recursos.
- **Aislamiento de los datos**: Al afectar a un solo microservicio, facilita realizar actualizaciones del esquema.

© JMA 2020. All rights reserved

Inconvenientes de los microservicios

- **Complejidad**: La arquitectura de microservicios es un estilo de programación distribuida. Cada servicio es más sencillo, pero el sistema como un todo es más complejo.
- **Desarrollo y pruebas**: Sobrecarga a los desarrolladores que deben implementar el mecanismo de comunicación entre servicios. Las herramientas existentes no siempre están diseñadas para trabajar con dependencias de servicios. La refactorización en los límites del servicio puede resultar difícil. También supone un desafío probar las dependencias de los servicios, especialmente cuando la aplicación está evolucionando rápidamente. La prueba es más difícil, requiere un mayor peso en las pruebas de integración.
- **Falta de gobernanza**: El enfoque descentralizado para la generación de microservicios tiene ventajas, pero también puede causar problemas, se puede acabar con tantos lenguajes y marcos de trabajo diferentes que la aplicación puede ser difícil de mantener.
- **Congestión y latencia de red**: El uso de muchos servicios pequeños y detallados puede dar lugar a más comunicación interservicios. Además, si la cadena de dependencias del servicio se hace demasiado larga, la latencia adicional puede constituir un problema. Tendrá que diseñar las APIs con atención. Hay que evitar que las APIs se comuniquen demasiado, pensar en formatos de serialización y buscar lugares para utilizar patrones de comunicación asincrónica.
- **Integridad de datos**: Cada microservicio es responsable de la conservación de sus propios datos y, como consecuencia, mantener la coherencia de los datos es un problema. Implementar casos de uso que abarcan múltiples servicios sin usar transacciones distribuidas es difícil.
- **Administración**: El mayor consumo de recursos, la complejidad del despliegue y la complejidad operativa de implementar y administrar un sistema que comprende muchos tipos componentes y servicios diferentes requiere una cultura de DevOps consolidada. El registro correlacionado entre servicios puede resultar un desafío.
- **Control de versiones**: Las actualizaciones de un servicio no deben interrumpir servicios que dependen de él. Es posible que varios servicios se actualicen en cualquier momento; por lo tanto, sin un cuidadoso diseño, podrían surgir problemas con la compatibilidad con versiones anteriores o posteriores.
- **Conjunto de habilidades**: Cada microservicio requiere una implementación completa, incluyendo interfaz de usuario, almacenamiento persistente y colaboración externa. En consecuencia, los equipos deben ser interdisciplinares, incluyendo toda la gama de habilidades.

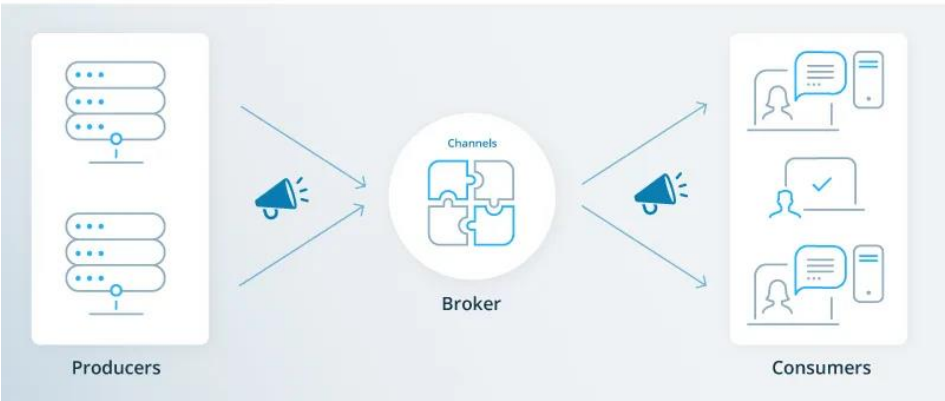
© JMA 2020. All rights reserved

Arquitectura basada en eventos

- Una arquitectura basada en eventos consta de productores de eventos que generan un flujo de eventos, y consumidores de eventos que escuchan los eventos. Los eventos se entregan casi en tiempo real, de modo que los consumidores pueden responder inmediatamente a los eventos cuando se producen. Los productores se desconectan de los consumidores, no saben si los consumidores están escuchando. Los consumidores también se desconectan entre sí, y cada consumidor ve todos los eventos.
- Una arquitectura basada en eventos puede usar un modelo pub/sub o un modelo de flujo de eventos.
  - Pub/sub: la infraestructura de mensajería mantiene el seguimiento de las suscripciones. Cuando se publica un evento, se envía el evento a cada suscriptor. Después de que se consume un evento, no se puede reproducir y los nuevos suscriptores no ven el evento.
  - Streaming de eventos: los eventos se escriben en un registro. Los eventos siguen un orden estricto (dentro de una partición) y son duraderos. Los clientes no se suscriben al flujo, sino que un cliente puede leer desde cualquiera de sus partes y es responsable de avanzar su posición en el flujo. Por lo que un cliente puede unirse en cualquier momento y puede reproducir los eventos.

© JMA 2020. All rights reserved

Arquitectura basada en eventos



© JMA 2020. All rights reserved

Arquitectura basada en eventos

- En el lado del consumidor, hay algunas variaciones comunes:
  - Procesamiento sencillo de eventos: Un evento desencadena inmediatamente una acción en el consumidor.
  - Procesamiento de eventos complejos: El consumidor procesa una serie de eventos, en busca de patrones en los datos de eventos.
  - Procesamiento de flujo de eventos: Los procesadores de flujos sirven para procesar o transformar el flujo. Puede haber varios procesadores de flujo para diferentes subsistemas de la aplicación.
- El origen de los eventos puede ser externo con respecto al sistema, como dispositivos físicos en una solución de IoT. En ese caso, el sistema debe ser capaz de ingerir los datos según el volumen y el rendimiento que requiere el origen de datos. En algunos sistemas, como IoT, los eventos se deben ingerir en volúmenes muy altos.
- En la práctica, es habitual tener varias instancias de un consumidor para evitar hacer que el consumidor se convierta en un único punto de error en el sistema. También podrían ser necesarias varias instancias para administrar el volumen y la frecuencia de los eventos.

© JMA 2020. All rights reserved

Arquitectura basada en eventos

- Esta arquitectura se debe utilizar para: Procesamiento de los mismos eventos en varios subsistemas , Procesamiento en tiempo real con retardo mínimo, Procesamiento de eventos complejos (como coincidencia de patrones o agregación durante ventanas de tiempo) y Procesamiento de un gran volumen y alta velocidad de datos (como IoT).
- Ventajas
  - Se desvinculan productores y consumidores.
  - No existen integraciones de punto a punto. Es fácil agregar nuevos consumidores al sistema.
  - Los consumidores pueden responder a eventos inmediatamente a medida que llegan.
  - Muy escalable y distribuida.
  - Los subsistemas tienen vistas independientes del flujo de eventos.
- Desafíos
  - Entrega garantizada: En algunos sistemas, especialmente en escenarios de IoT, es fundamental garantizar la entrega de los eventos.
  - Procesamiento de eventos en orden o exactamente solo una vez: Cada tipo de consumidor normalmente se ejecuta en varias instancias, a fin de conseguir resiliencia y escalabilidad. Esto puede suponer un desafío si se deben procesar los eventos en orden (dentro de un tipo de consumidor) o si la lógica de procesamiento no es idempotente.

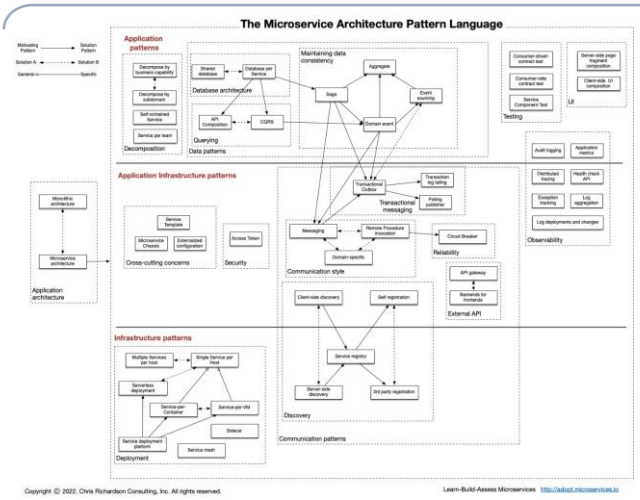
© JMA 2020. All rights reserved

Arquitectura basada en eventos

- Esta arquitectura se debe utilizar para: Procesamiento de los mismos eventos en varios subsistemas , Procesamiento en tiempo real con retardo mínimo, Procesamiento de eventos complejos (como coincidencia de patrones o agregación durante ventanas de tiempo) y Procesamiento de un gran volumen y alta velocidad de datos (como IoT).
- Ventajas
  - Se desvinculan productores y consumidores.
  - No existen integraciones de punto a punto. Es fácil agregar nuevos consumidores al sistema.
  - Los consumidores pueden responder a eventos inmediatamente a medida que llegan.
  - Muy escalable y distribuida.
  - Los subsistemas tienen vistas independientes del flujo de eventos.
- Desafíos
  - Entrega garantizada: En algunos sistemas, especialmente en escenarios de IoT, es fundamental garantizar la entrega de los eventos.
  - Procesamiento de eventos en orden o exactamente solo una vez: Cada tipo de consumidor normalmente se ejecuta en varias instancias, a fin de conseguir resiliencia y escalabilidad. Esto puede suponer un desafío si se deben procesar los eventos en orden (dentro de un tipo de consumidor) o si la lógica de procesamiento no es idempotente.

© JMA 2020. All rights reserved

Patrones arquitectónicos y de diseño



- La arquitectura de microservicios no es una panacea. Tiene varios inconvenientes. Además, al utilizar esta arquitectura, existen numerosos problemas que debe abordar.
- El lenguaje de patrones de la arquitectura de microservicios es una colección de patrones para aplicar la arquitectura de microservicios. Tiene dos objetivos:
  - El lenguaje de patrones le permite decidir si los microservicios son adecuados para su aplicación.
  - El lenguaje de patrones le permite usar la arquitectura de microservicios con éxito.
- Referencias:
  - https://microservices.io/
  - https://www.enterpriseintegrationpatterns.com/patterns/messaging/
  - https://learn.microsoft.com/es-es/azure/architecture/patterns/

© JMA 2020. All rights reserved

De Monolito a Microservicios

- Patrones para descomponer monolitos
  - Descomposición por capacidad empresarial
  - Descomposición por subdominio
  - Descomposición por transacciones
  - Descomposición por equipo
- Patrones para migrar de monolito a microservicios
  - Reescritura Big Bang
  - Strangler Fig
  - Branch by Abstraction
  - Anti-corruption Layer
  - Ambassador

© JMA 2020. All rights reserved

Preocupaciones transversales

- En referencia a las APIs, servicios y microservicios, la tendencia natural es a crecer, tanto por nuevas funcionalidades del sistema como por escalado horizontal.
- Todo ello provoca una serie de preocupaciones adicionales:
  - Localización de los servicios.
  - Balanceo de carga.
  - Tolerancia a fallos.
  - Gestión de la configuración.
  - Gestión de logs.
  - Gestión de los despliegues.
  - y otras …

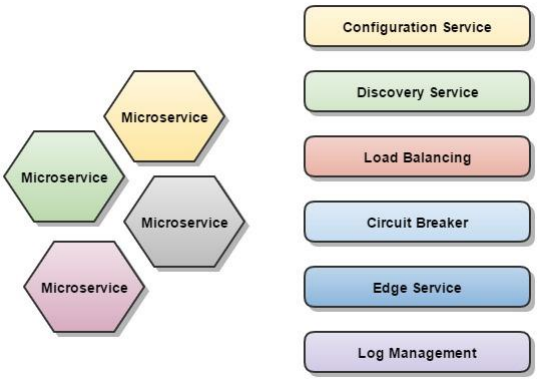
© JMA 2020. All rights reserved

Implantación

- Para la implantación de una arquitectura basada en APIs hemos tener en cuenta 3 aspectos principalmente:
  - Un modelo de referencia en el que definir las necesidades de una arquitectura de las APIs.
  - Un modelo de implementación en el que decidir y concretar la implementación de los componentes vistos en el modelo de referencia.
  - Un modelo de despliegue donde definir cómo se van a desplegar los distintos componentes de la arquitectura en los diferentes entornos.

© JMA 2020. All rights reserved

Modelo de referencia



© JMA 2020. All rights reserved

Modelo de referencia

- Servidor perimetral / exposición de servicios (Edge server)
  - Será un gateway en el que se expondrán los servicios a consumir.
- Servicio de registro / descubrimiento
  - Este servicio centralizado será el encargado de proveer los endpoints de los servicios para su consumo. Todo microservicio, en su proceso de arranque, se registrará automáticamente en él.
- Balanceo de carga (Load balancer)
  - Este patrón de implementación permite el balanceo entre distintas instancias de forma transparente a la hora de consumir un servicio.
- Tolerancia a fallos (Circuit breaker)
  - Mediante este patrón conseguiremos que cuando se produzca un fallo, este no se propague en cascada por todo el pipe de llamadas, y poder gestionar el error de forma controlada a nivel local del servicio donde se produjo.
- Mensajería:
  - Las invocaciones siempre serán síncronas (REST, SOAP, …) o también llamadas asíncronas (AMQP).

© JMA 2020. All rights reserved

Modelo de referencia

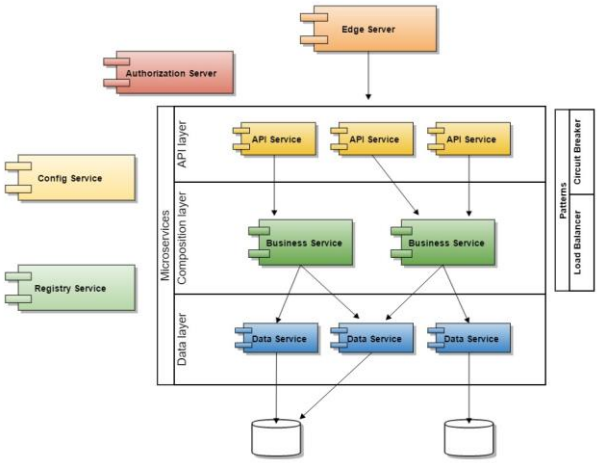
- Servidor de configuración central
  - Este componente se encargará de centralizar y proveer remotamente la configuración a cada API. Esta configuración se mantiene convencionalmente en un repositorio Git, lo que nos permitirá gestionar su propio ciclo de vida y versionado.
- Servidor de autenticación / autorización
  - Para implementar la capa de seguridad (recomendable en la capa de servicios API)
- Centralización de logs
  - Se hace necesario un mecanismo para centralizar la gestión de logs. Pues sería inviable la consulta de cada log individual de cada uno de los microservicios.
- Monitorización
  - Para poder disponer de mecanismos y dashboard para monitorizar aspectos de los nodos como, salud, carga de trabajo...

© JMA 2020. All rights reserved

Modelo de referencia

© JMA 2020. All rights reserved

Modelo de referencia



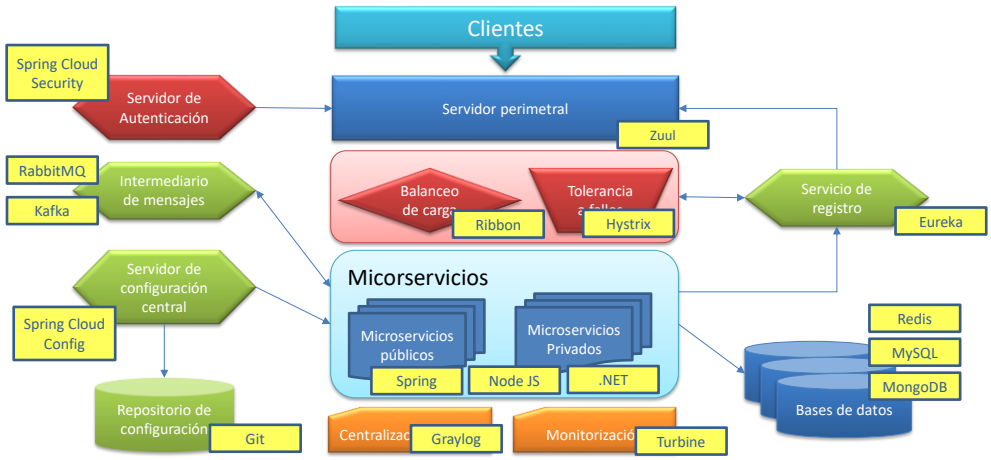
© JMA 2020. All rights reserved

Modelo de implementación (Netflix OSS)

- Basándonos en el modelo de referencia, vamos a definir un modelo de implementación para cada uno de los componentes descritos. Para ello podemos hacer uso del stack tecnológico de Spring Cloud y Netflix OSS:
  - Microservicios propiamente dichos: Serán aplicaciones Spring Boot con controladores Spring MVC. Se puede utilizar Swagger para documentar y definir nuestra API.
  - Config Server: microservicio basado en Spring Cloud Config y se utilizará Git como repositorio de configuración.
  - Registry / Discovery Service: microservicio basado en Eureka de Netflix OSS.
  - Load Balancer: se puede utilizar Ribbon de Netflix OSS que ya viene integrado en REST-template de Spring.
  - Circuit breaker: se puede utilizar Hystrix de Netflix OSS.
  - Gestión de Logs: se puede utilizar Graylog
  - Servidor perimetral: se puede utilizar Zuul de Netflix OSS.
  - Servidor de autenticación / autorización: se puede utilizar el servicio con Spring Cloud Security.
  - Agregador de métricas: se puede utilizar el servicio Turbine.
  - Intermediario de mensajes: se puede utilizar AMQP con RabbitMQ o Kafka.

© JMA 2020. All rights reserved

Modelo de implementación (Netflix OSS)



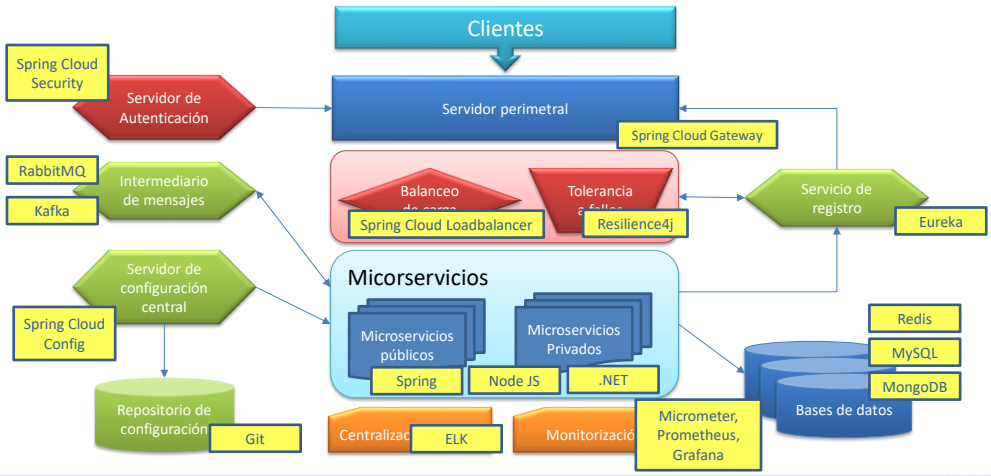
© JMA 2020. All rights reserved

Modelo de implementación (Spring Cloud)

- Basándonos en el modelo de referencia, vamos a definir un modelo de implementación para cada uno de los componentes descritos. Para ello podemos hacer uso del stack tecnológico de Spring Cloud y Netflix OSS:
  - Microservicios propiamente dichos: Serán aplicaciones Spring Boot con controladores Spring MVC. Se puede utilizar Swagger para documentar y definir nuestra API.
  - Config Server: microservicio basado en Spring Cloud Config y se utilizará Git como repositorio de configuración.
  - Registry / Discovery Service: microservicio basado en Eureka de Netflix OSS.
  - Load Balancer: se puede utilizar Spring Cloud Loadbalancer que ya viene integrado en REST-template de Spring.
  - Circuit breaker: se puede utilizar Spring Cloud Circuit Breaker con Resilience4j.
  - Gestión de Logs: se puede utilizar Elasticsearch, Logstash y Kibana
  - Servidor perimetral: se puede utilizar Spring Cloud Gateway.
  - Servidor de autenticación / autorización: se puede utilizar el servicio con Spring Cloud Security.
  - Agregador de métricas: se puede utilizar el servicio Turbine.
  - Intermediario de mensajes: se puede utilizar AMQP con RabbitMQ o Kafka.

© JMA 2020. All rights reserved

Modelo de implementación (Spring Cloud)



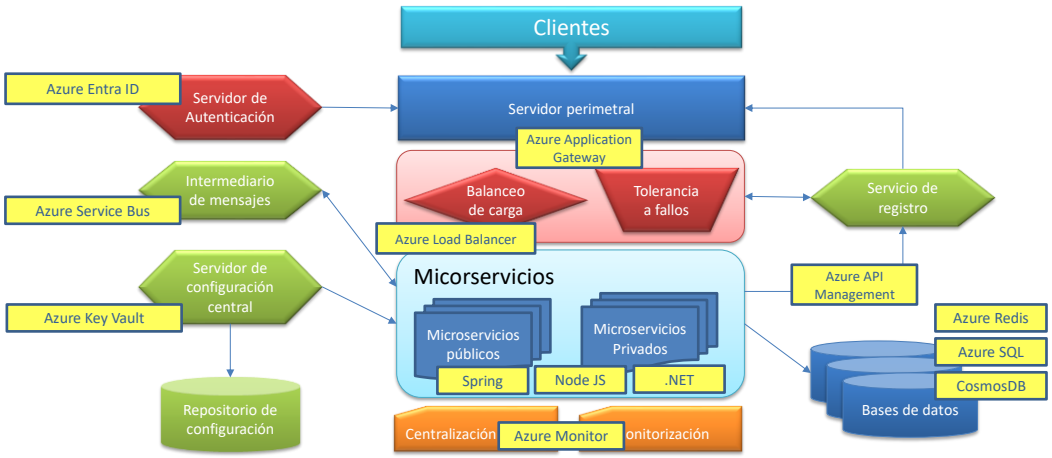
© JMA 2020. All rights reserved

Modelo de implementación (Azure)

- Basándonos en el modelo de referencia, vamos a definir un modelo de implementación para cada uno de los componentes descritos. Para ello podemos hacer uso del stack tecnológico de suministrado por Azure:
  - Microservicios propiamente dichos: Serán aplicaciones ASP.NET Core con WebApi. Se puede utilizar OpenAPI para documentar y definir nuestra API.
  - Azure Key Vault: Se puede utilizar para almacenar de forma segura y controlar de manera estricta el acceso a los tokens, contraseñas, certificados, claves de API y otros secretos.
  - Azure API Management: es una solución completa para publicar API para clientes externos e internos.
    - Servidor perimetral, Registry / Discovery Service, Load Balancer (con Azure Application Gateway), Circuit breaker.
  - Servidor de autorización: Azure Entra ID (Azure Active Directory) es un servicio de administración de identidades y acceso basado en la nube de Microsoft.
  - Azure Monitor: ayuda a maximizar la disponibilidad y el rendimiento de las aplicaciones y los servicios.
    - Agregador de métricas: Detección y diagnóstico de problemas en aplicaciones y dependencias con Application Insights.
    - Gestión de Logs: Profundización en sus datos de supervisión con Log Analytics para la solución de problemas y diagnósticos profundos.
  - Intermediario de mensajes: se puede utilizar AMQP con Azure Service Bus.

© JMA 2020. All rights reserved

Modelo de implementación (Azure)



© JMA 2020. All rights reserved

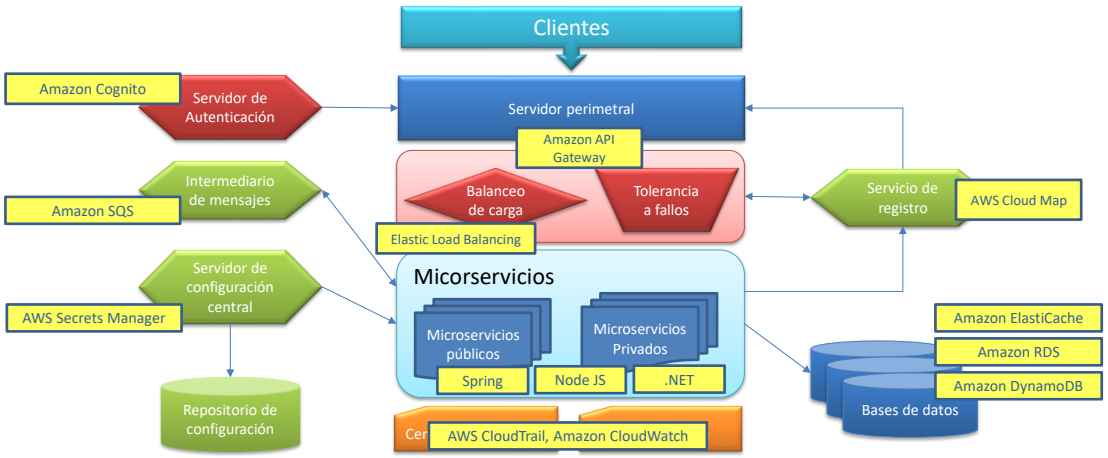
Modelo de implementación (AWS)

- Basándonos en el modelo de referencia, vamos a definir un modelo de implementación para cada uno de los componentes descritos. Para ello podemos hacer uso del stack tecnológico de Amazon Web Services:
  - Amazon API Gateway: Proxy de la API
  - Elastic Load Balancing: Balanceador de carga de aplicaciones
  - AWS Cloud Map: Detección de servicios
  - Amazon RDS: Bases de datos relacionales
  - Amazon DynamoDB: Bases de datos NoSQL
  - Amazon ElastiCache: Almacenamiento en caché
  - Amazon Simple Queue Service (Amazon SQS): Colas de mensajes
  - AWS CloudTrail, Amazon CloudWatch: Monitorización de API
  - Amazon Cognito, AWS IAM: Registro, inicio de sesión y control de acceso de usuarios
  - AWS Secrets Manager, AWS KMS: Datos confidenciales de configuración

https://aws.amazon.com/es/microservices/

© JMA 2020. All rights reserved

Modelo de implementación (AWS)



© JMA 2020. All rights reserved

Modelo de despliegue

- El modelo de despliegue hace referencia al modo en que vamos a organizar y gestionar los despliegues de los microservicios, así como a las tecnologías que podemos usar para tal fin.
- El despliegue de los microservicios es una parte primordial de esta arquitectura. Muchas de las ventajas que aportan, como la escalabilidad, son posibles gracias al sistema de despliegue.
- Existen convencionalmente varios patrones en este sentido a la hora de encapsular microservicios:
  - Máquinas virtuales.
  - Contenedores.
  - Sin servidor: FaaS (Functions-as-a-Service)
- Los microservicios están íntimamente ligados al concepto de contenedores (una especie de máquinas virtuales ligeras que corren de forma independiente, pero utilizando directamente los recursos del host en lugar de un SO completo). Hablar de contenedores es hablar de Docker. Con este software se pueden crear las imágenes de los contenedores para después crear instancias a demanda.

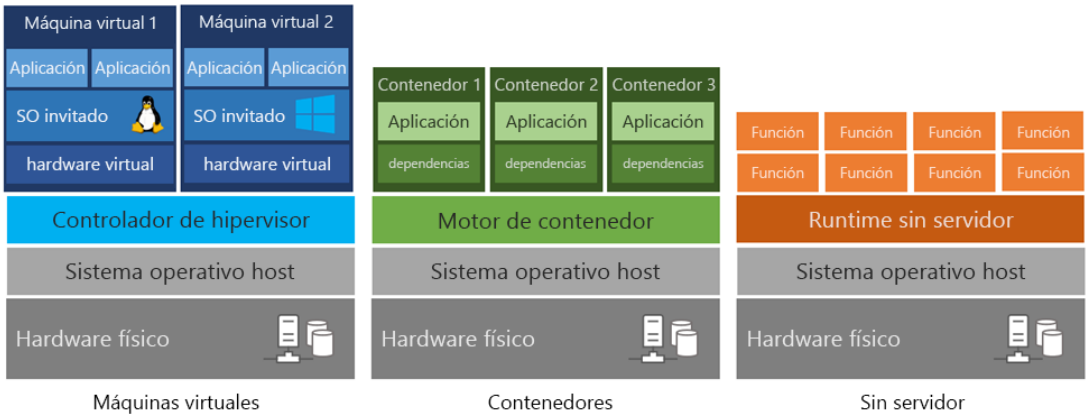
© JMA 2020. All rights reserved

Modelo de despliegue

- Las imágenes Docker son como plantillas. Constan de un conjunto de capas y cada una aporta un conjunto de software a lo anterior, hasta construir una imagen completa.
- Por ejemplo, podríamos tener una imagen con una capa Ubuntu y otra capa con un servidor LAMP. De esta forma tendríamos una imagen para ejecutar como servidor PHP.
- Las capas suelen ser bastante ligeras. La capa de Ubuntu, por ejemplo, contiene algunos los ficheros del SO y otros, como el Kernel, los toma del host.
- Los contenedores toman una imagen y la ejecutan, añadiendo una capa de lectura/escritura, ya que las imágenes son de sólo lectura.
- Dada su naturaleza volátil (el contenedor puede parar en cualquier momento y volver a arrancarse otra instancia), para el almacenamiento se usan volúmenes, que están fuera de los contenedores.

© JMA 2020. All rights reserved

Contenedores



© JMA 2020. All rights reserved

Modelo de despliegue

- Sin embargo, esto no es suficiente para dotar a nuestro sistema de una buena escalabilidad. El siguiente paso será pensar en la automatización y orquestación de los despliegues siguiendo el paradigma cloud. Se necesita una plataforma que gestione los contenedores, y para ello existen soluciones como Kubernetes.
- Kubernetes permite gestionar grandes cantidades de contenedores, agrupándolos en pods. También se encarga de gestionar servicios que estos necesitan, como conexiones de red y almacenamiento, entre otros. Además, proporciona también esta parte de despliegue automático, que puede utilizarse con sus componentes o con componentes de otras tecnologías como Spring Cloud+Netflix OSS.
- Todavía se puede dar una vuelta de tuerca más, incluyendo otra capa por encima de Docker y Kubernetes: Openshift. En este caso estamos hablando de un PaaS que, utilizando Docker y Kubernetes, realiza una gestión más completa y amigable de nuestro sistema de microservicios. Por ejemplo, nos evita interactuar con la interfaz CLI de Kubernetes y simplifica algunos procesos. Además, nos provee de más herramientas para una gestión más completa del ciclo de vida, como construcción, test y creación de imágenes. Incluye los despliegues automáticos como parte de sus servicios y, en sus últimas versiones, el escalado automático.
- Openshift también proporciona sus propios componentes, que de nuevo pueden mezclarse con los de otras tecnologías.

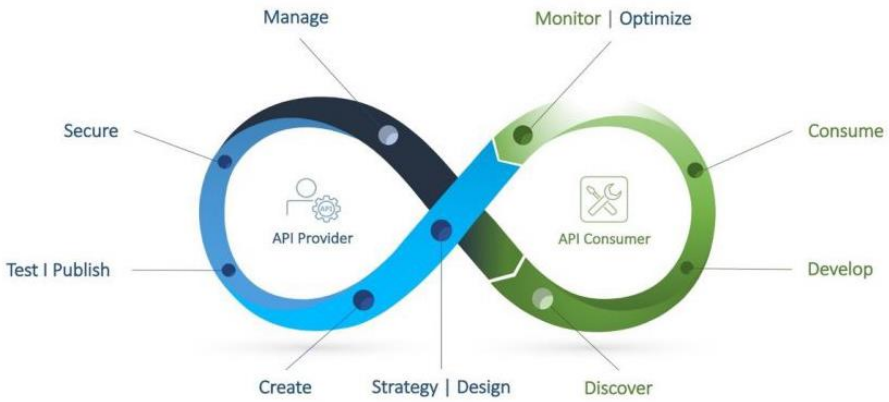
© JMA 2020. All rights reserved

FaaS (Functions-as-a-Service)

- El auge de la informática sin servidor es una de las innovaciones más importantes de la actualidad. Las tecnologías sin servidor, como Azure Functions, AWS Lambda o Google Cloud Functions, permiten a los desarrolladores centrarse por completo en escribir código. Toda la infraestructura informática de la que dependen (máquinas virtuales (VM), compatibilidad con la escalabilidad y demás) se administra por ellos. Debido a esto, la creación de aplicaciones se vuelve más rápida y sencilla. Ejecutar dichas aplicaciones a menudo resulta más barato, porque solo se le cobra por los recursos informáticos que realmente usa el código.
- La arquitectura serverless habilita la ejecución de una aplicación mediante contenedores efímeros y sin estado; estos son creados en el momento en el que se produce un evento que dispare dicha aplicación. Contrariamente a lo que nos sugiere el término, serverless no significa «sin servidor», sino que éstos se usan como un elemento anónimo más de la infraestructura, apoyándose en las ventajas del cloud computing.
- La tecnología sin servidor apareció por primera vez en lo que se conoce como tecnologías de plataforma de aplicaciones como servicio (aPaaS), actualmente como FaaS (Functions-as-a-Service).

© JMA 2020. All rights reserved

Ciclo de vida



© JMA 2020. All rights reserved

Ciclo de vida

- Estrategia: que camino se va a seguir y como se planifica
- Creación: una vez se tenga una estrategia y un plan sólidos, es hora de crear las APIs.
- Pruebas: antes de publicar, es importante completar las pruebas de API para garantizar que cumplan con las expectativas de rendimiento, funcionalidad y seguridad.
- Publicación: una vez probado, es hora de publicar la API para que estén disponibles para los desarrolladores.
- Protección: los riesgos y las preocupaciones de seguridad son un problema común en la actualidad.
- Administración: una vez publicadas, los creadores deben administrar y mantener las APIs para asegurarse de que estén actualizadas y que la integridad de sus APIs no se vea comprometida.
- Integración: cuando se ofrece las APIs para consumo público o privado, la documentación es un componente importante para que los desarrolladores comprendan las capacidades clave.
- Monitorización: una vez las APIs están activas, es necesario supervisarlas y analizar los datos para detectar anomalías o detectar nuevas necesidades.
- Promoción: hay varias formas de comercializar las APIs, incluida su inclusión en un mercado de APIs.
- Monetización: se puede optar por ofrecer las APIs de forma gratuita o, cuando existe la oportunidad, se puede monetizar las APIs y generar ingresos adicionales para el negocio.
- Retirada: Retirar las APIs es la última etapa del ciclo de vida de una API y ocurre por una variedad de razones, incluidos cambios tecnológicos y preocupaciones de seguridad.

© JMA 2020. All rights reserved

Ciclo de vida

© JMA 2020. All rights reserved

¿Qué es una API?

- API es el acrónimo de Application Programming Interface, que es un intermediario de software que permite que dos aplicaciones se comuniquen entre sí.
- A lo largo de los años, lo que es una API a menudo se ha descrito como cualquier tipo de interfaz de conectividad genérica para una aplicación. Más recientemente, sin embargo, una API moderna ha adquirido algunas características que las hacen extraordinariamente valiosas y útiles:
  - Las API modernas se adhieren a los estándares (generalmente HTTP y REST), que son amigables para los desarrolladores, de fácil acceso y comprensibles ampliamente
  - Se tratan más como productos que como código. Están diseñados para el consumo de audiencias específicas, están documentados y están versionadas de manera que los usuarios puedan tener ciertas expectativas sobre su mantenimiento y ciclo de vida.
  - Debido a que están mucho más estandarizados, tienen una disciplina mucho más sólida para la seguridad y la gobernanza, además de monitorear y administrar el rendimiento y la escalabilidad.
  - Como cualquier otra pieza de software producido, una API moderna tiene su propio ciclo de vida de desarrollo de software (SDLC) de diseño, prueba, construcción, administración y control de versiones.

© JMA 2020. All rights reserved

API Strategy

- Una empresa debe desarrollar una estrategia de API que consista en APIs tanto públicas como privadas. Cuando una empresa lanza APIs públicas que potencian las aplicaciones orientadas al consumidor, habilita nuevas formas de interactuar y conectarse con sus clientes a través de aplicaciones web, móviles y sociales. Al desarrollar APIs privadas, las empresas pueden ofrecer a sus empleados y socios nuevas herramientas que les ayuden a agilizar las operaciones y servir a los clientes aún mejor. En este entorno dinámico, a medida que más y más empresas crean e incorporan APIs, es cada vez más crítico que las empresas innovadoras desarrollen y ejecuten estrategias API de éxito.
- Como ejemplo de los beneficios que una API Strategy puede aportar a una organización, alrededor de 2002, Jeffrey Preston Bezos, director ejecutivo de Amazon, envió un correo a sus empleados con los siguientes puntos:
  - Todos los equipos expondrán sus datos y funcionalidad a través de interfaces de servicios.
  - Los equipos deben comunicarse entre sí a través de estas interfaces.
  - No se permitirá otra forma de comunicación: ni vinculación directa, ni acceso directo a bases de datos de otros equipos, ni memoria compartida ni utilización de ningún tipo de puerta trasera. Sólo se permitirán comunicaciones a través de llamadas que utilicen interfaces de red.
  - La tecnología empleada por cada equipo no debe ser un problema.
  - Todas las interfaces de los servicios, sin excepción, deben ser diseñadas con el objetivo de ser externalizables. Esto es, el equipo debe planear y diseñar sus interfaces para los desarrolladores del resto del mundo. Sin excepciones.
- El correo finalizaba de la siguiente manera: “Todo aquel que no siga las directrices será despido. Gracias, ¡pasad un buen día!”. Desde hace ya varios años Amazon es el primer proveedor IaaS mundial distanciado significativamente de sus competidores.

© JMA 2020. All rights reserved

API Economy

- El ecosistema de APIs especifica de qué manera el uso de estas micro aplicaciones por terceros puede beneficiar económicamente a una organización, bien por reducción de costes o bien por alquiler o venta de sus propios desarrollos:
  - API as a Service: Obtención de beneficios mediante la exposición de APIs de servicios que son valiosos para terceros y están dispuestos a pagar por su uso.
  - API Products: Desarrollo de herramientas encargadas de facilitar la exposición e integración de aplicaciones a través de sus APIs.
- Una API Economy es, en definitiva, un servicio basado en API que demuestra algún tipo de rentabilidad al negocio, ya sea económica o estratégicamente. Es fundamental pensar en la API como un producto. La economía está cambiando gracias a que las APIs abren nuevos canales, tanto de ingresos como de innovación.
- En general existen muchos servicios APIs de terceros que permiten a un negocio escalar rápidamente para crear productos finales con una inversión y riesgo mínimo.
- Una empresa puede cambiar su estrategia de ventas a la comercialización como proveedor de servicios APIs a terceros: los recursos aquí son sus datos y servicios. La API Economy es un facilitador para convertir una empresa u organización en una plataforma.

© JMA 2020. All rights reserved

Tipos de API por propósito

- Es raro que una organización decida que necesita una API de la nada; la mayoría de las veces, las organizaciones comienzan con una idea, aplicación, innovación o caso de uso que requiere conectividad a otros sistemas o conjuntos de datos. Las APIs entran en escena como un medio para permitir la conectividad entre los sistemas y los conjuntos de datos que deben integrarse.
- Las organizaciones pueden implementar APIs para muchos propósitos: desde exponer internamente la funcionalidad de un sistema central hasta habilitar una aplicación móvil orientada al cliente. El marco de conectividad incluye:
  - APIs del sistema: las APIs del sistema desbloquean datos de los sistemas centrales de registro dentro de una organización. Los ejemplos de sistemas críticos de los que las API podrían desbloquear datos incluyen ERP, sistemas de facturación, CRM y bases de datos.
  - APIs de proceso: las APIs de proceso interactúan y dan forma a los datos dentro de un solo sistema o entre sistemas, rompiendo los silos de datos. Las APIs de proceso proporcionan un medio para combinar datos y organizar varias APIs del sistema para un propósito comercial específico. Algunos ejemplos de esto incluyen la creación de una vista de 360 grados del cliente, el cumplimiento del pedido y el estado del envío.
  - APIs de experiencia: las APIs de experiencia proporcionan un contexto empresarial para los datos y procesos que se desbloquearon y establecieron con las APIs de proceso y sistema. Las APIs de experiencia exponen los datos para que los consuma su público objetivo; esto funciona en un amplio conjunto de canales en una variedad de formas. Algunos ejemplos son las aplicaciones móviles, los portales internos para los datos del cliente o un sistema de cara al cliente que rastrea las entregas.

© JMA 2020. All rights reserved

Tipos de API por estrategias de gestión

- Una vez que se haya determinado el caso de uso de las APIs en la organización, es hora de determinar quién accederá a estas APIs. La mayoría de las veces, el caso de uso y el usuario previsto van de la mano; por ejemplo, es posible que desee mostrar los datos del cliente para sus agentes de ventas y servicios internos; el usuario final previsto, en este caso, son los empleados internos.
- Los tres tipos de APIs según cómo se administran y quién accede a ellas son:
  - APIs externas: Los terceros, que son externos a la organización, pueden acceder a las APIs externas . A menudo, hacen que los datos y servicios de una organización sean fácilmente accesibles en autoservicio por desarrolladores de todo el mundo que buscan crear aplicaciones e integraciones innovadoras.
  - APIs internas: Las APIs internas son lo opuesto a las APIs abiertas, ya que no son accesibles para los consumidores externos y solo están disponibles para los desarrolladores internos de una organización. Las APIs internas pueden permitir iniciativas en toda la empresa, desde la adopción de DevOps y arquitecturas de microservicios hasta la modernización heredada y la transformación digital. El uso y la reutilización de estas APIs pueden mejorar la productividad, la eficiencia y la agilidad de una organización.
  - APIs de socios: Las APIs de socios se encuentran en algún lugar entre las APIs internas y externas. Son APIs a las que acceden otras personas ajenas a la organización con permisos exclusivos. Por lo general, este acceso especial se otorga a terceros específicos para facilitar una asociación comercial estratégica. Un caso de uso común de una API de socio es cuando dos organizaciones desean compartir datos entre sí, se configuraría una API de socio para que cada organización tenga acceso a los datos necesarios con el conjunto correcto de credenciales y permisos.

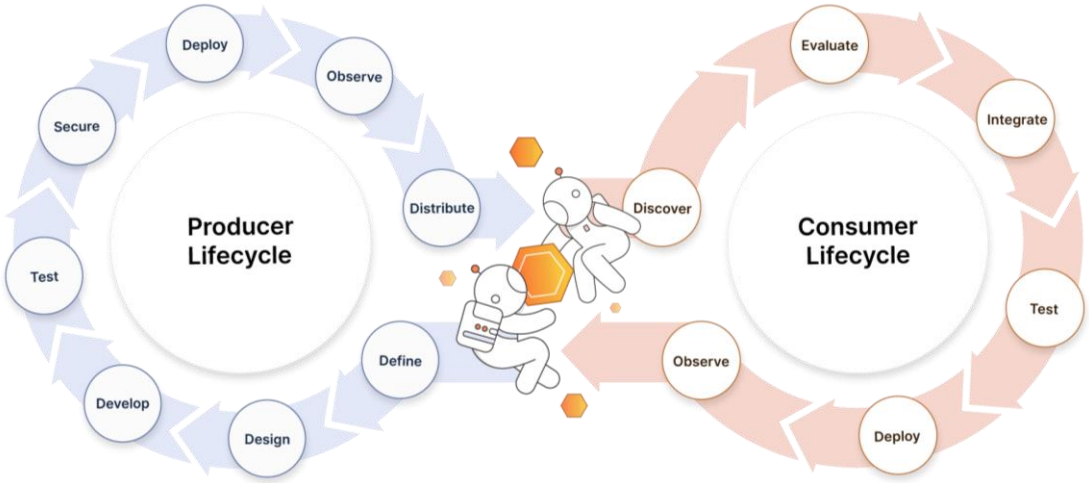
© JMA 2020. All rights reserved

API First

- El enfoque basado en API First significa que, para cualquier proyecto de desarrollo dado, las APIs se tratan como "ciudadanos de primera clase": que todo sobre un proyecto gira en torno a la idea de que el producto final es un conjunto de APIs consumido por las aplicaciones del cliente.
- El enfoque de API First implica que los desarrollos de APIs sean consistentes y reutilizables, lo que se puede lograr mediante el uso de un lenguaje formal de descripción de APIs para establecer un contrato sobre cómo se supone que se comportará la API. Establecer un contrato implica pasar más tiempo pensando en el diseño de una API.
- A menudo también implica una planificación y colaboración adicionales con las partes interesadas, proporcionando retroalimentación de los consumidores sobre el diseño de una API antes de escribir cualquier código evitando costosos errores.
- Entre sus ventajas se encuentran:
  - Los equipos de desarrollo pueden trabajar en paralelo.
  - Reduce el coste de desarrollar aplicaciones
  - Aumenta la velocidad de desarrollo.
  - Asegura buenas experiencias de desarrollador
  - Reduce el riesgo de fallos
  - Proporcionar un sólido perímetro de seguridad

© JMA 2020. All rights reserved

Ciclo de vida de las API



© JMA 2020. All rights reserved

https://www.postman.com/api-first/

http://spring.io

SPRING CON SPRING BOOT

© JMA 2020. All rights reserved

Spring

- Spring Framework es un marco de desarrollo de aplicaciones Java que facilita la creación de aplicaciones empresariales robustas y modulares al proporcionar un conjunto integral de funcionalidades, como inversión de control, contenedor de beans o gestión de transacciones. Utilizar Spring y Spring Boot proporciona ventajas como la flexibilidad, la modularidad, la escalabilidad y una amplia gama de características integradas que aceleran el desarrollo de aplicaciones empresariales.
- Inicialmente era un ejemplo hecho para el libro “J2EE design and development” de Rod Johnson en 2003, que defendía alternativas a la “visión oficial” de aplicación JavaEE basada en EJBs. Actualmente es un framework open source que facilita el desarrollo de aplicaciones java JEE & JSE (no esta limitado a aplicaciones Web ni a java).
- Provee de un contenedor encargado de manejar el ciclo de vida de los objetos (beans) para que los desarrolladores se enfoquen a la lógica de negocio. Permite integración con diferentes frameworks. Surge como una alternativa a EJB’s
- Actualmente es un framework completo compuesto por múltiples módulos/proyectos que cubre todas las capas de la aplicación, con decenas de desarrolladores y miles de descargas al día.

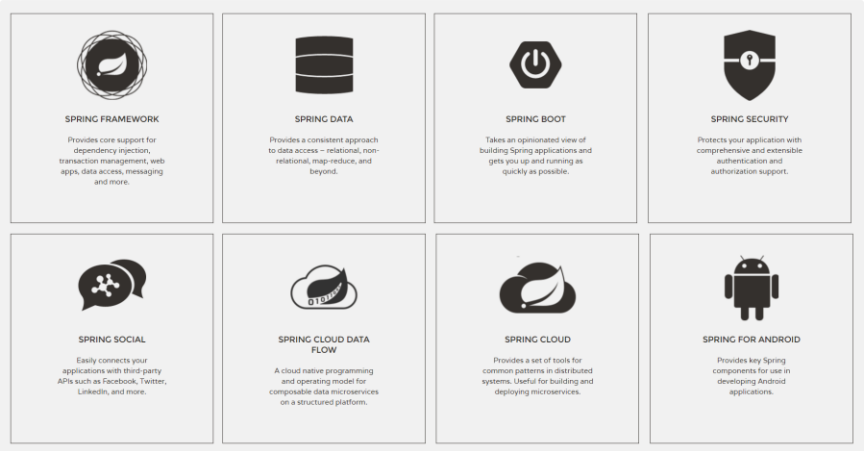
© JMA 2020. All rights reserved

Características

- Ligero
  - No se refiere a la cantidad de clases sino al mínimo impacto que se tiene al integrar Spring.
- No intrusivo
  - Generalmente los objetos que se programan no tienen dependencias de clases específicas de Spring
- Flexible
  - Aunque Spring provee funcionalidad para manejar las diferentes capas de la aplicación (vista, lógica de negocio, acceso a datos) no es necesario usarlo para todo. Brinda la posibilidad de utilizarlo en la capa o capas que queramos.
- Multiplataforma
  - Escrito en Java, corre sobre JVM

© JMA 2020. All rights reserved

Proyectos



© JMA 2020. All rights reserved

Módulos necesarios

- Spring Framework
  - Spring Core
    - Contenedor IoC (inversión de control) - inyector de dependencia
  - Spring MVC
    - Framework basado en MVC para aplicaciones web y servicios REST
- Spring Data
  - Simplifica el acceso a los datos: JPA, bases de datos relacionales / NoSQL, nube
- Spring Boot
  - Simplifica el desarrollo de Spring: inicio rápido con menos codificación

© JMA 2020. All rights reserved

Spring Boot

- Spring tiene una gran cantidad de módulos que implican multitud de configuraciones. Estas configuraciones pueden requerir mucho tiempo, pueden ser desconocidas para principiantes y suelen ser repetitivas.
- La solución de Spring es Spring Boot, que aplica el concepto de Convention over Configuration (CoC).
- CoC es un paradigma de programación que minimiza las decisiones que tienen que tomar los desarrolladores, simplificando tareas.
- No obstante, la flexibilidad no se pierde, ya que a pesar de otorgar valores por defecto, siempre se puede configurar de forma extendida.
- De esta forma se evita la repetición de tareas básicas a la hora de construir un proyecto.
- Spring Boot es una herramienta que nace con la finalidad de simplificar aun más el desarrollo de aplicaciones basadas en el framework Spring Core: que el desarrollador solo se centre en el desarrollo de la solución, olvidándose por completo de la compleja configuración que actualmente tiene Spring Core para poder funcionar.

© JMA 2020. All rights reserved

Spring Boot

- Resolución de dependencias:
  - Con Spring Boot solo hay que determinar que tipo de proyecto estaremos utilizando y el se encarga de resolver todas las librerías/dependencias para que la aplicación funcione.
- Configuración:
  - Spring Boot cuenta con un complejo módulo que autoconfigura todos los aspectos de nuestra aplicación para poder simplemente ejecutar la aplicación, sin tener que definir absolutamente nada.
- Despliegue:
  - Spring Boot se puede ejecutar como una aplicación Stand-alone, pero también es posible ejecutar aplicaciones web, ya que es posible desplegar las aplicaciones mediante un servidor web integrado, como es el caso de Tomcat, Jetty o Undertow.

© JMA 2020. All rights reserved

Spring Boot

- Métricas:
  - Por defecto, Spring Boot cuenta con servicios que permite consultar el estado de salud de la aplicación, permitiendo saber si la aplicación está encendida o apagada, memoria utilizada y disponible, número y detalle de los Bean's creado por la aplicación, controles para el prendido y apagado, etc.
- Extensible:
  - Spring Boot permite la creación de complementos, los cuales ayudan a que la comunidad de Software Libre cree nuevos módulos que faciliten aún más el desarrollo.
- Productividad:
  - Herramientas de productividad para desarrolladores como Spring Initializr, Lombok, LiveReload y Auto Restart, funcionan en su IDE favorito: Spring Tool Suite, IntelliJ IDEA y NetBeans.

© JMA 2020. All rights reserved

Dependencias: starters

- Los starters son un conjunto de descriptores de dependencias convenientes (versiones compatibles, ya probadas) que se pueden incluir en la aplicación.
- Se obtiene una ventanilla única para el módulo de Spring y la tecnología relacionada que se necesita, sin tener que buscar a través de códigos de ejemplo y copiar/pegar cargas de descriptores de dependencia.
- Por ejemplo, si desea comenzar a utilizar Spring con JPA para un acceso CRUD a base de datos, basta con incluir la dependencia spring-boot-starter-data-jpa en el proyecto.

© JMA 2020. All rights reserved

devtools

- Al realizar nuevos cambios en nuestra aplicación, podemos hacer que el arranque se reinicie automáticamente. Para eso es necesario incluir una dependencia Maven extra: spring-boot-devtools.
- Durante el tiempo de ejecución, Spring Boot supervisa la carpeta que se encuentra en classpath (en maven, las carpetas que están en la carpeta "target"). Solo necesitamos activar la compilación de las fuentes en los cambios que causarán la actualización de la carpeta 'destino' y Spring Boot reiniciará automáticamente la aplicación. Si estamos utilizando Eclipse IDE, la acción de guardar puede desencadenar la compilación.
- El módulo spring-boot-devtools incluye un servidor LiveReload incorporado que activa una actualización del navegador cuando se cambia un recurso. Las extensiones del navegador LiveReload están disponibles gratuitamente para Chrome, Firefox y Safari desde:
  http://livereload.com/extensions/
- Para configurar LiveReload:
  spring.devtools.restart.additional-paths=META-INF/resources/**
  spring.devtools.livereload.enabled=true # por defecto

© JMA 2020. All rights reserved

Spring Tools

https://spring.io/tools

- Spring Tools 4 es la próxima generación de herramientas Spring para los entorno IDE de codificación favoritos. Proporciona soporte de primera clase para el desarrollo de aplicaciones empresariales basadas en Spring, ya sea que se prefiera Eclipse, Visual Studio Code o Theia IDE.
  - Help → Eclipse Marketplace …
    - Spring Tools 4 for Spring Boot
- Spring Tool Suite (STS) es un IDE basado en la versión Java EE de Eclipse, pero altamente personalizable para trabajar con Spring Framework.
  - IDE gratuito, personalización del Eclipse

© JMA 2020. All rights reserved

Crear proyecto

- Desde web:
  - https://start.spring.io/
  - Descomprimir en el workspace
  - Import → Maven → Existing Maven Project
- Desde Eclipse:
  - New Project → Sprint Boot → Spring Started Project
- Dependencias
  - Web
  - JPA
  - JDBC (o proyecto personalizado)

© JMA 2020. All rights reserved

Application

```
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication implements CommandLineRunner {

	public static void main(String[] args) {
		SpringApplication.run(ApiHrApplication.class, args);
	}

	@Override
	public void run(String... args) throws Exception {
		// Opcional: Procesar los args una vez arrancado SprintBoot
	}

	@Bean
	CommandLineRunner lookup() {
		return args -> { log.info("Aplicación arrancada"); };
	}
}
```

© JMA 2020. All rights reserved

Configuración

- @Configuration: Indica que esta es una clase usada para configurar el contenedor Spring.
- @ComponentScan: Escanea los paquetes de nuestro proyecto en busca de los componentes que hayamos creado, ellos son, las clases que utilizan las siguientes anotaciones: @Component, @Service, @Controller, @Repository.
- @EnableAutoConfiguration: Habilita la configuración automática, esta herramienta analiza el classpath y el archivo application.properties para configurar nuestra aplicación en base a las librerías y valores de configuración encontrados, por ejemplo: al encontrar el motor de bases de datos H2 la aplicación se configura para utilizar este motor de datos, al encontrar Thymeleaf se crearan los beans necesarios para utilizar este motor de plantillas para generar las vistas de nuestra aplicación web.
- @SpringBootApplication: Es el equivalente a utilizar las anotaciones: @Configuration, @EnableAutoConfiguration y @ComponentScan

© JMA 2020. All rights reserved

Configuración

- Editar src/main/resources/application.properties:

```
server.port=${PORT:8080}
# Oracle settings
spring.datasource.url=jdbc:oracle:thin:@localhost:1521:xe
spring.datasource.username=hr
spring.datasource.password=hr
spring.datasource.driver-class=oracle.jdbc.driver.OracleDriver

# MySQL settigs
spring.datasource.url=jdbc:mysql://localhost:3306/sakila
spring.datasource.username=root
spring.datasource.password=root
#spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} %-5level %logger{36} - %msg%n
spring.jpa.show-sql=true
```

- Repetir con src/test/resources/application.properties
- Eclipse: Run Configurations → Arguments → VM Arguments: -DPORT=8888

© JMA 2020. All rights reserved

Instalación de MySQL

- Descargar e instalar:
  - https://mariadb.org/download/
- Incluir en la seccion [mysqld] de %MYSQL_ROOT%/data/my.ini
  - default_time_zone='+01:00'
- Descargar bases de datos de ejemplos:
  - https://dev.mysql.com/doc/index-other.html
- Instalar bases de datos de ejemplos:
  - mysql -u root -p < employees.sql
  - mysql -u root -p < sakila-schema.sql
  - mysql -u root -p < sakila-data.sql

© JMA 2020. All rights reserved

Instalación con Docker

- Docker Toolbox
  - Windows 10 Pro ++: https://docs.docker.com/docker-for-windows/install/
  - Otras: https://github.com/docker/toolbox/releases
- Ejecutar Docker QuickStart
- Para crear el contenedor de MySQL con la base de datos Sakila:
  - docker run -d --name mysql-sakila -e MYSQL_ROOT_PASSWORD=root -p 3306:3306 jamarton/mysql-sakila
- Para crear el contenedor de MongoDB:
  - docker run -d --name mongodb -p 27017:27017 mongo
- Para crear el contenedor de Redis:
  - docker run --name redis -p 6379:6379 -d redis
  - docker run --name redis-insight -d -v redisinsight:/db -p 6380:8001 redislabs/redisinsight:latest
  - docker run -d --name redis-commander -p 8081:8081 rediscommander/redis-commander:latest
- Para crear el contenedor de RabbitMQ:
  - docker run -d --hostname rabbitmq --name rabbitmq -p 4369:4369 -p 5671:5671 -p 5672:5672 -p 15671:15671 -p 15672:15672 -p 25672:25672 -e RABBITMQ_DEFAULT_USER=admin -e RABBITMQ_DEFAULT_PASS=curso rabbitmq:management-alpine

© JMA 2020. All rights reserved

ESTILOS DE COMUNICACIÓN

© JMA 2020. All rights reserved

Falacias de la computación distribuida

- La red es confiable.
- La latencia es cero.
- El ancho de banda es infinito.
- La red es segura.
- La topología de la red no cambia.
- Sólo hay un administrador del sistema de red.
- El coste del transporte es cero.
- La red es homogénea.

© JMA 2020. All rights reserved

Estilos de comunicación

- El cliente y los servicios, o los servicios entre si, pueden comunicarse a través de muchos tipos diferentes de comunicación, cada uno destinado a un escenario y unos objetivos distintos. Inicialmente, estos tipos de comunicaciones se pueden clasificar por diferentes criterios.
- El primer criterio define si el proceso es sincrónico o asincrónico:
  - Proceso síncrono: HTTP es el protocolo síncrono mas utilizado. El cliente envía una solicitud y espera una respuesta del servicio, solo puede continuar su tarea cuando recibe la respuesta del servidor. Es independiente de la ejecución de código de cliente, que puede ser síncrono (el subproceso está bloqueado) o asíncrono (el subproceso no está bloqueado y la respuesta dispara una devolución de llamada).
  - Proceso asíncrono: Otros protocolos como AMQP (un protocolo compatible con muchos sistemas operativos y entornos de nube) usan mensajes asíncronos. Normalmente el código de cliente o el remitente del mensaje no espera ninguna respuesta. Simplemente se envía el mensaje a una cola de un agente de mensajes, que son escuchadas por los consumidores.
- El segundo criterio define si la comunicación tiene un único receptor o varios:
  - Receptor único 1:1 (comando, punto a punto, P2P): Cada solicitud debe ser procesada por un receptor o servicio exactamente (como en el patrón Command).
  - Varios receptores 1:N (eventos, publicación/subscripción, Pub/Sub): Cada solicitud puede ser procesada por entre cero y varios receptores. Este tipo de comunicación debe ser asíncrona (basada en un bus de eventos o un agente de mensajes).

© JMA 2020. All rights reserved

Criterio según su base

- Basados en Recursos: Los servicios exponen información, documentos que incluyen tanto identificadores de datos como de acciones (enlaces y formularios), las operaciones CRUD están predefinidas. REST es un estilo arquitectónico que separa las preocupaciones del consumidor y del proveedor de la API al depender de comandos que están integrados en el protocolo de red subyacente. REST (Representational State Transfer) es extremadamente flexible en el formato de sus cargas útiles de datos, lo que permite una variedad de formatos de datos populares como JSON y XML, entre otros.
- Basados en Procedimientos: Las llamadas a procedimiento remoto, o RPC, generalmente requieren que los desarrolladores ejecuten bloques específicos de código en otro sistema: operaciones. RPC es independiente del protocolo, lo que significa que tiene el potencial de ser compatible con muchos protocolos, pero también pierde los beneficios de usar capacidades de protocolo nativo (por ejemplo, almacenamiento en caché). La utilización de diferentes estándares da como resultado un acoplamiento más estrecho entre los consumidores y los proveedores de API y las tecnologías implicadas, lo que a su vez sobrecarga a los desarrolladores involucrados en todos los aspectos de un ecosistema de APIs impulsado por RPC. Los patrones de arquitectura de RPC se pueden observar en tecnologías API populares como SOAP, GraphQL y gRPC.
- Basados en Eventos/Streaming: a veces denominadas arquitecturas de eventos, en tiempo real, de transmisión, asíncronas o push, las APIs impulsadas por eventos no esperan a que un consumidor de la API las llame antes de entregar una respuesta. En cambio, una respuesta se desencadena por la ocurrencia de un evento. Estos servicios exponen eventos, con una información mínima, a los que los clientes pueden suscribirse para recibir actualizaciones cuando cambian los valores del servicio. Hay un puñado de variaciones para este estilo que incluyen (entre otras) reactivo, publicador/suscriptor, notificación de eventos y CQRS.

© JMA 2020. All rights reserved

Patrón: Invocación a procedimiento remoto (RPI)

- Motivación:
  - Ha aplicado el patrón de Microservicios. Los servicios deben manejar las solicitudes de los clientes de la aplicación. Además, los servicios a veces deben colaborar para manejar esas solicitudes. Deben utilizar un protocolo de comunicación entre procesos.
- Intención:
  - ¿Cómo se van a comunicar los servicios?
- Requisitos:
  - Los servicios están escritos en diferentes tecnologías
  - Los clientes, a su vez, utilizan diferentes tecnologías. Hay aplicaciones web del lado del cliente (AJAX/WebSokets)
- Solución:
  - Utilizar RPI para la comunicación entre servicios. El cliente utiliza un protocolo basado en solicitud/respuesta para realizar solicitudes a un servicio. Los servicios públicos utilizaran REST.

© JMA 2020. All rights reserved

Patrón: Invocación a procedimiento remoto (RPI)

- Implementación:
  - Crear servicios REST con Spring Boot y Spring MVC con Spring HATEOAS, Spring Data Rest, …
- Consecuencias:
  - Este patrón tiene los siguientes beneficios:
    - Sencillo y familiar
    - El patrón solicitud/respuesta síncrona es fácil
    - Sistema más sencillo ya que no requiere componentes intermediarios.
  - Este patrón tiene los siguientes inconvenientes:
    - Por lo general, solo admite la solicitud/respuesta y no otros patrones de interacción, como notificaciones, solicitud/respuesta asíncrona, publicación/suscripción, publicación/respuesta asíncrona
    - Disponibilidad reducida ya que el cliente y el servicio deben estar disponibles durante la interacción
    - El cliente necesita descubrir ubicaciones de instancias de servicio.
- Patrones relacionados:
  - La mensajería es un patrón alternativo.
  - La configuración externalizada proporciona la ubicación de red (lógica).
  - Un cliente debe utilizar el descubrimiento del lado del cliente y el descubrimiento del lado del servidor para localizar una instancia de servicio
  - Normalmente, un cliente usará el patrón cortocircuito (interruptor automático) para mejorar la confiabilidad

© JMA 2020. All rights reserved

Patrón: Mensajería

- Motivación:
  - Ha aplicado el patrón de Microservicios. Los servicios deben manejar las solicitudes de los clientes de la aplicación. Además, los servicios a veces deben colaborar para manejar esas solicitudes. Deben utilizar un protocolo de comunicación entre procesos.
- Intención:
  - ¿Cómo se van a comunicar los servicios?
- Requisitos:
  - Los servicios están escritos en diferentes tecnologías
  - Los clientes, a su vez, utilizan diferentes tecnologías
- Solución:
  - Utilizar la mensajería asíncrona para la comunicación entre servicios. Servicios que se comunican mediante el intercambio de mensajes a través de canales de mensajería. El estándar AMQP (Advanced Message Queuing Protocol) es un protocolo estándar abierto en la capa de aplicaciones de un sistema de comunicación. Esta ampliamente difundido y soportado. Sus características son la orientación a mensajes, encolamiento ("queuing"), enrutamiento (tanto punto-a-punto como publicación-subscripción), exactitud y seguridad.

© JMA 2020. All rights reserved

Patrón: Mensajería

- Implementación:
  - Requiere un intermediario de mensajes: RabbitMQ, Apache Kafka
  - Crear servicios con Spring Boot con Spring for RabbitMQ, …
- Consecuencias:
  - Este patrón tiene los siguientes beneficios:
    - Bajo acoplamiento en tiempo de ejecución ya que desacopla el remitente del mensaje del consumidor
    - Disponibilidad mejorada desde que el intermediario de mensajes almacena los mensajes en búfer hasta que el consumidor puede procesarlos
    - Admite una variedad de patrones de comunicación que incluyen solicitud/respuesta síncrona y asíncrona, notificaciones, publicación/suscripción, publicación/respuesta asíncrona, etc.
  - Este patrón tiene los siguientes inconvenientes:
    - Mayor complejidad de los procesos asíncronos frente a los secuenciales síncronos.
    - Complejidad adicional, el intermediario de mensajes debe ser otro componente del sistema que debe instalarse y configurarse. También deberá ser replicado para disponibilidad y capacidad.
- Patrones relacionados:
  - La Invocación a procedimiento remoto (RPI) mensajería es un patrón alternativo.
  - La configuración externalizada proporciona los nombres de los canales de mensajes (lógicos) y la ubicación del intermediario de mensajes.
  - El patrón Saga y el patrón CQRS usan mensajes
  - El patrón de Bandeja de salida transaccional permite que los mensajes se envíen como parte de una transacción de base de datos

© JMA 2020. All rights reserved

Evolución histórica de los servicios

- Precursores:
  - RPC: Llamadas a Procedimientos Remotos
  - Binarios: CORBA, Java RMI, .NET Remoting
  - XML-RPC: Precursor del SOAP
- Actuales:
  - Servicios Web XML o Servicios SOAP
  - Servicios Web REST o API REST
    - WebHooks
  - Servicios GraphQL
  - Servicios gRPC

AÑO	Descripción
1976	Aparición de RPC (Remote Procedure Call) en Sistema Unix
1990	Aparición de DCE (Distributed Computing Environment) que es un sistema de software para computación distribuida, basado en RPC.
1991	Aparición de Microsoft RPC basado en DCE para sistemas Windows.
1992	Aparición de DCOM (Microsoft) y CORBA (ORB) para la creación de componentes software distribuidos.
1997	Aparición de Java RMI en JDK 1.1
1998	Aparición de XML-RPC
1999	Aparición de SOAP 1.0, WSDL, UDDI
2000	Definición del REST
2012	Propuesta de GraphQL por Facebook
2015	Desarrollo de gRPC por Google

© JMA 2020. All rights reserved

Servicios SOAP

- SOAP es un protocolo de comunicación web altamente estandarizado basado en formatos de tipo texto en XML. Publicado por Microsoft en 1999, un año después de XML-RPC, SOAP heredó mucho de él.
- Basado en operaciones, de tipos texto, en formato XML y comunicaciones síncronas.
- Ventajas:
  - Independiente del lenguaje y la plataforma: La funcionalidad incorporada para crear servicios basados en web permite a SOAP manejar las comunicaciones y hacer que las respuestas sean independientes del lenguaje y la plataforma.
  - Vinculado a una variedad de protocolos de transporte: SOAP es flexible en términos de protocolos de transferencia para adaptarse a múltiples escenarios.
  - Manejo de errores incorporado: La especificación de la API SOAP permite devolver el mensaje Retry XML con el código de error y su explicación.
  - Varias extensiones de seguridad: Integrado con los protocolos WS-Security, SOAP cumple con una calidad de transacción de nivel empresarial. Proporciona privacidad e integridad dentro de las transacciones al tiempo que permite el cifrado a nivel de mensaje.

© JMA 2020. All rights reserved

Servicios SOAP

- Inconvenientes:
  - Solamente acepta formato XML: Los mensajes SOAP contienen una gran cantidad de metadatos y solo admiten estructuras XML detalladas para solicitudes y respuestas.
  - De peso pesado: Debido al gran tamaño de los archivos XML, los servicios SOAP requieren un gran ancho de banda hasta para la información mas nimia.
  - Conocimientos estrictamente especializados: La creación de servidores de API SOAP requiere un conocimiento profundo de todos los protocolos involucrados y sus reglas altamente restringidas o disponer de framework que simplifiquen su utilización que no están disponibles en todas las plataformas y lenguajes.
  - Actualización de mensajes tediosa: Al requerir un esfuerzo adicional para agregar o eliminar las propiedades del mensaje, el esquema SOAP rígido ralentiza la adopción.

© JMA 2020. All rights reserved

Servicios REST

- REpresentational State Transfer es un estilo arquitectónico autoexplicativo basado en el uso del protocolo HTTP e hipermedia y establece un conjunto de restricciones arquitectónicas y destinado a una amplia adopción. Definido en el 2000 por Roy Fielding, para no reinventar la rueda, se basa en aprovechar lo que ya estaba definido en el HTTP pero que no se utilizaba. RESTful hace referencia a un servicio web que implementa la arquitectura REST.
- Restringe la forma de usar de las URLs, los métodos (verbos) de HTTP, sus encabezados (Accept, Content-type, …) y sus códigos de estado.
- Basado en recursos, independiente de formato (texto o binario) y comunicaciones síncronas.
- Ventajas:
  - Soporte de múltiples formatos: La capacidad de admitir múltiples formatos (a menudo JSON y XML) para almacenar e intercambiar datos es una de las razones por las que REST es actualmente una opción predominante para crear APIs públicas.
  - Documentación mínima, basados en convenios y el descubrimiento hipermedia.
  - Mínima curva de aprendizaje: usa tecnologías ampliamente conocidas: HTTP, URL, MIME, …

© JMA 2020. All rights reserved

Servicios REST

- Inconvenientes:
  - Uso de HTTP: Aunque es la infraestructura mas ampliamente difundida y utilizada, está restringido a ella.
  - Grandes cargas útiles: REST devuelve una gran cantidad de metadatos enriquecidos para que el cliente pueda comprender todo lo necesario sobre el estado de la aplicación solo a partir de sus respuestas.
  - Sin estructura REST única: No existe una forma exacta y correcta de crear una API REST. Cómo modelar los recursos y qué recursos modelar dependerá de cada escenario. Esto hace que REST sea simple en teoría, pero difícil en la práctica.
  - Problemas de recuperación excesiva o insuficiente: Devolver todo suele conllevar demasiados datos y el convenio no estable mecanismos de filtrado, paginación o proyecciones. El uso de hipermedia para obtener datos relacionados requiere solicitudes adicionales y encadenadas.
  - Convenio genérico: En muchos casos no acorde con las reglas de negocio (create o replace, delete, …) lo que acaba requiriendo documentación.

© JMA 2020. All rights reserved

Servicios GraphQL

- GraphQL es una sintaxis que describe cómo obtener la descripción del modelo de datos y cómo realizar una solicitud de datos precisa, consultas y mutaciones, pensada para los modelos de datos con muchas entidades complejas que hacen referencia entre sí. Fue propuesta por Facebook en 2012, publicada en 2015 y posteriormente pasada a open source en 2018.
- Basado en consultas, en formato JSON y comunicaciones síncronas.
- Ventajas:
  - Un esquema de GraphQL establece una fuente única de información, ofrece una forma de unificar todo en un único servicio.
  - Las llamadas a GraphQL se gestionan mediante HTTP POST en un solo recorrido de ida y vuelta. Los clientes obtienen lo que solicitan sin que se genere una sobrecarga.
  - Los tipos de datos bien definidos reducen los problemas de comunicación entre el cliente y el servidor. Un cliente puede solicitar una lista de los tipos de datos disponibles y esto es ideal para la generación automática de documentación.
  - GraphQL permite que las APIs de las aplicaciones evolucionen sin afectar a las consultas actuales.
  - GraphQL no exige una arquitectura de aplicación específica. Puede incorporarse sobre una API de REST actual y funcionar con las herramientas de gestión de APIs actuales. Hay muchas extensiones open source de GraphQL que ofrecen características que no están disponibles con las APIs de REST.

© JMA 2020. All rights reserved

Servicios GraphQL

- Inconvenientes:
  - GraphQL intercambia complejidad por flexibilidad. Tener demasiados campos anidados en una solicitud puede provocar una sobrecarga del sistema. Además, delega gran parte del trabajo de las consultas de datos en el servidor, lo cual representa una mayor complejidad para los desarrolladores de servidores.
  - GraphQL no indica cómo almacenar los datos ni qué lenguaje de programación utilizar, requiere disponer de framework que simplifiquen su utilización que no están disponibles en todas las plataformas.
  - Como GraphQL no utiliza la semántica de almacenamiento en caché HTTP, requiere un esfuerzo de almacenamiento en caché personalizado.
  - GraphQL presenta una curva de aprendizaje elevada para desarrolladores que tienen experiencia con las APIs de REST.

© JMA 2020. All rights reserved

Servicios gRPC

- gRPC es un marco de llamada a procedimiento remoto (RPC) de alto rendimiento e independiente de lenguaje. Desarrollado por Google en el año 2015, y luego convertido en código abierto. Como GraphQL, es una especificación que se implementa en una variedad de lenguajes.
- Basado en contratos, en el formato binario Protocol Buffers y comunicaciones síncronas/asíncronas.
- Las principales ventajas de gRPC son:
  - Marco de RPC moderno, ligero y de alto rendimiento.
  - La especificación gRPC es preceptiva con respecto al formato que debe seguir un servicio gRPC, formaliza un contrato independiente de lenguaje.
  - Es compatible con el intercambio de datos bidireccional y asíncrono al estar basado en HTTP/2, así como con la compresión, multiplexación y streaming.
  - Uso reducido de red al usar un formato de mensaje binario eficaz que genera cargas de mensajes pequeñas.

© JMA 2020. All rights reserved

Servicios gRPC

- Inconvenientes:
  - Tanto el cliente como el servidor deben admitir la misma especificación de búfer de protocolo, requiere un estricto control de versiones. gRPC es un formato muy particular que proporciona una ejecución ultrarrápida a expensas de la flexibilidad.
  - Basado en HTTP/2 que no tiene soporte universal para interacciones cliente-servidor de cara al público general en Internet y una compatibilidad limitada con navegadores.
  - gRPC es una especificación, por lo que requiere disponer de framework que permitan su utilización tanto en el cliente como en le servidor y que no están disponibles en todas las plataformas y lenguajes.
  - El proceso de serialización/deserialización para su conversión a binario es costoso en términos de CPU.
  - gRPC presenta una curva de aprendizaje mas empinada que la de REST.

© JMA 2020. All rights reserved

Tecnología de clientes web (Navegadores)

- JavaScript Asíncrono + XML (AJAX) no es una tecnología por sí misma, es un término que describe un nuevo modo de utilizar conjuntamente varias tecnologías existentes. Esto incluye: HTML o XHTML, CSS, JavaScript, DOM, XML, XSLT, y lo más importante, el objeto XMLHttpRequest (fetch). Cuando estas tecnologías se combinan en un modelo AJAX, es posible lograr aplicaciones web capaces de actualizarse continuamente sin tener que volver a cargar la página completa.
- WebSockets es una tecnología avanzada que hace posible abrir una sesión de comunicación bidireccional entre el navegador y un servidor: se puede enviar mensajes a un servidor y recibir respuestas controladas por eventos sin tener que consultar al servidor para una respuesta.
- Server Sent Events (SSE) es una tecnología basada en streaming HTTP que pretenden estandarizar la comunicación COMET (long polling) en los navegadores cuyo flujo de comunicación sólo va desde el servidor hacia el cliente (no es bidireccional como los WebSockets). La idea consiste en que el cliente crea una conexión con el servidor una sola vez y este le va enviando información (eventos) al cliente cuando hay nuevos datos.
- ASP.NET Signalr es una biblioteca para desarrolladores de ASP.NET que simplifica el proceso de agregar funcionalidad web en tiempo real: la posibilidad de que el código de servidor inserte el contenido en los clientes conectados al instante a medida que esté disponible sin espera a nuevas solicitudes. Signalr usa WebSocket cuando está disponible o recurre las otras cuando es necesario.

© JMA 2020. All rights reserved

WebHooks

- Los webhooks son eventos que desencadenan acciones. Su nombre se debe a que funcionan como «ganchos» de los programas en Internet y casi siempre se utilizan para la comunicación entre sistemas. Son la manera más sencilla de obtener un aviso cuando algo ocurre en otro sistema y para el intercambio de datos entre aplicaciones web.
- Un webhook es una retro llamada HTTP, una solicitud HTTP POST insertada en una página web, que interviene cuando ocurre algo (una notificación de evento a través de HTTP POST).
- Los webhooks se utilizan para las notificaciones en tiempo real (con los datos del evento en JSON o XML) a una determinada dirección http:// o https://, que puede:
  - almacenar los datos del evento en JSON o XML
  - generar una respuesta que permita actualizarse al sistema donde se produce el evento
  - ejecutar un proceso en el sistema receptor del evento (Ej: enviar un correo electrónico)
- Los webhook están pensados para su utilización desde páginas web y sus diferentes consumidores: navegadores, correo electrónico, webapps, …
- Un ejemplo típico es su utilización en correos electrónico de marketing para notificar al servidor que debe enviar un nuevo correo electrónico porque el usuario a abierto el mensaje.
- Pueden considerarse una versión especializada y simplificada de los servicios REST (solo POST).

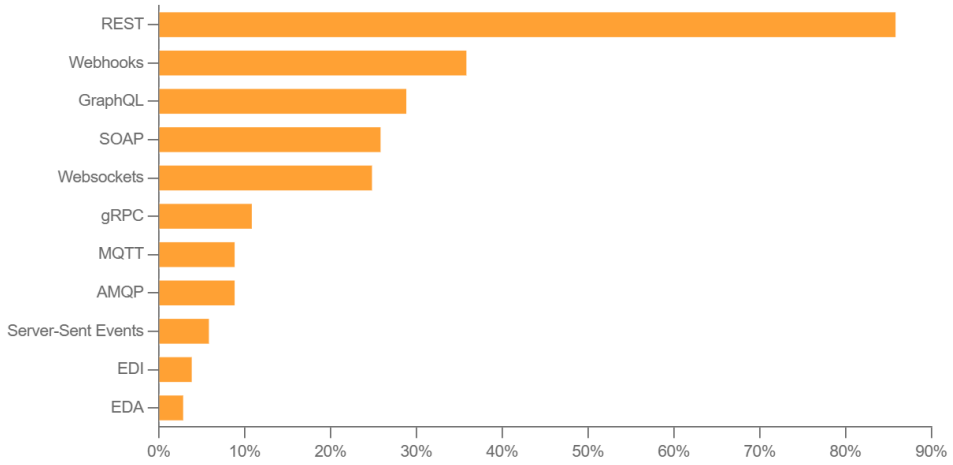
© JMA 2020. All rights reserved

Protocolos y Estándares

- Síncronos:
  - SOAP: Basado en operaciones, de tipos texto, en formato XML y comunicaciones síncronas.
  - REST: Basado en recursos, independiente de formato (texto/binario) y comunicaciones síncronas.
  - GraphQL: Basado en consultas, en formato JSON y comunicaciones síncronas.
  - gRPC: Basado en contratos, en el formato binario Protocol Buffers y comunicaciones síncronas/asíncronas.
- Asíncronos:
  - WebSockets: protocolo estándar abierto, elemental, texto y binario.
  - STOMP (Simple/Streaming Text Oriented Messaging Protocol): protocolo estándar abierto, basado en texto, soluciones simples y ligero.
  - AMQP (Advanced Message Queuing Protocol): protocolo estándar abierto, binario, encolamiento, P2P y Pub/Sub, exactitud y seguridad
  - MQTT (Message Queue Telemetry Transport): protocolo estándar abierto, binario, P2P, liviano, soluciones simples y seguridad
  - JMS (Java Message Service): API, binario Java, P2P y Pub/Sub

© JMA 2020. All rights reserved

Estilos arquitectónicos mas utilizados



© JMA 2020. All rights reserved

https://www.postman.com/state-of-api/api-technologies/#api-technologies

Estilos de comunicación

MENSAJERÍA

© JMA 2020. All rights reserved

Mensajería y contextos de dominio

- En términos de integración, cuando se construyen estructuras de comunicación entre los diferentes procesos, es común ver productos y enfoques que ponen el énfasis de la inteligencia en el mecanismo de comunicación en sí.
- Normalmente los productos de ESB (Enterprise Service Bus) son un ejemplo donde generalmente se incluyen sofisticadas estructuras para el enrutado de mensajes, la coreografía, la transformación, la aplicación de reglas de negocio, etc.
- En términos de diseño, también es común ver que a la hora de definir dónde colocamos cierto código que representa reglas de negocio, a veces, decidimos hacerlo dentro del ESB por conveniencia o rapidez. Entonces, convertimos al ESB en un elemento de integración clave para el funcionamiento del proceso en su totalidad y, esa excesiva inteligencia puesta en un único componente (la tubería), torna al sistema en algo más frágil que antes.

© JMA 2020. All rights reserved

Mensajería y contextos de dominio

- Las aplicaciones creadas con microservicios pretenden ser tan disociadas y cohesivas como sean posible, ellas poseen su propia lógica de dominio y actúan más como filtros en el clásico sentido Unix:
  - recibe una solicitud, aplica la lógica apropiada y produce una respuesta.
  - estos pasos son coordinados utilizando protocolos REST simples en lugar de protocolos complejos WS-BPEL o la coordinación por una herramienta central.
- Los dos protocolos que se utilizan con mayor frecuencia son:
  - Petición/respuesta de HTTP con recursos API
  - Mensajería liviana, como puede ser RabbitMQ o ActiveMQ.
- Estos principios y protocolos hacen que los equipos de microservicios, a la hora de integrar servicios mas complejos, prefieran el concepto de coreografía, en lugar de orquestación.
- En orquestación, contamos con un cerebro central para orientar y conducir el proceso, al igual que el director de una orquesta.
- Con coreografía, le informamos a cada parte del sistema de su trabajo y se les deja trabajar en los detalles, como los bailarines en un ballet que se encuentran en su camino y reaccionan ante otros a su alrededor.

© JMA 2020. All rights reserved

Mensajería y contextos de dominio

- La desventaja del enfoque de orquestación es que el orquestador se puede convertir en una autoridad de gobierno central demasiado fuerte y pasar a ser un punto central donde toda la lógica gira alrededor de él.
- En cambio, con un enfoque coreografiado, podríamos tener un servicio emitiendo un mensaje asincrónico, los servicios interesados sólo se suscriben a esos eventos y reaccionan en consecuencia.
- Este enfoque es significativamente más desacoplado. Si algún nuevo servicio está interesado en recibir los mensajes, simplemente tiene que subscribirse a los eventos y hacer su trabajo cuando sea necesario.
- La desventaja de la coreografía es que la vista explícita del proceso de negocio del modelado de proceso ahora sólo se refleja de manera implícita en el sistema.
- Esto implica que se necesita de trabajo adicional para asegurarse de que alguien pueda controlar y realizar un seguimiento de que hayan sucedido las cosas correctas.
- Para solucionar este problema, normalmente es necesario crear un sistema de monitoreo que refleje explícitamente la vista del proceso de negocio del modelado de procesos, pero que a su vez, haga un seguimiento de lo que cada uno de los servicios realiza como entidad independiente que le permite ver excepciones mapeadas al flujo de proceso más explícito.

© JMA 2020. All rights reserved

Comandos vs Eventos

- Los mensajes se pueden clasificar en dos categorías principales:
  - Si el productor espera una acción del consumidor, ese mensaje es un comando.
  - Si el mensaje informa a los consumidor de que se ha realizado una acción, dicho mensaje es un evento.
- El productor envía un comando con la intención de que un consumidor realicen una operación dentro del ámbito de una transacción de negocio. Un comando es un mensaje de alto valor y debe entregarse al menos una vez. Si se pierde un comando, puede producirse un error en toda la transacción de negocio. Además, un comando no debe procesarse más de una vez. Si eso sucede, podría producirse una transacción errónea. Un cliente puede recibir pedidos o cargos duplicados. Los comandos se suelen usar para administrar el flujo de trabajo de una transacción de negocio de varios pasos. En función de la lógica de negocios, el productor puede esperar que el consumidor confirme el mensaje e informe de los resultados de la operación. En función de ese resultado, el productor puede elegir el plan de acción adecuado.
- Un evento es un tipo de mensaje que el productor genera para anunciar datos. El productor (conocido como publicador en este contexto) no espera que los eventos produzcan ninguna acción. Los consumidores interesados pueden suscribirse, escuchar eventos y tomar medidas en función de su escenario de consumo. Los eventos pueden tener varios suscriptores o ninguno. Dos suscriptores diferentes pueden reaccionar a un evento con diferentes acciones sin detectar al otro. El productor y el consumidor están acoplados de forma flexible y se administran de forma independiente. No se espera que el consumidor confirme el evento para el productor. Un consumidor que ya no está interesado en los eventos, puede finalizar la suscripción. El consumidor se quita de la canalización sin afectar al productor ni a la funcionalidad general del sistema.

© JMA 2020. All rights reserved

https://www.rabbitmq.com/

RABBITMQ

© JMA 2020. All rights reserved

Colas de mensajes

- En determinadas ocasiones, los servicios deben integrarse con otros actores, componentes o sistemas internos y externos, siendo necesario aportar o recibir información de ellos. En la mayoría de los casos, estas comunicaciones tienen que estar permanentemente disponibles, ser rápidas, seguras, asíncronas y fiables entre otros requisitos.
- Las colas de mensajes (MQ) solucionan estas necesidades, actuando de intermediario entre emisores y destinatarios, o en un contexto más definido, productores y consumidores de mensajes. Se pueden usar para reducir las cargas y los tiempos de entrega por parte de los servicios, ya que las tareas, que normalmente tardarían bastante tiempo en procesarse, se pueden delegar a un tercero cuyo único trabajo es realizarlas.
- El uso de colas de mensajes también es bueno cuando se desea distribuir un mensaje a múltiples destinatarios. Además aportan otros beneficios como:
  - Garantía de entrega y orden: Los mensajes se consumen, en el mismo orden que se llegaron a la cola, y son consumidos una única vez
  - Redundancia: Las colas persisten los mensajes hasta que son procesados por completo
  - Desacoplamiento: Siendo capas intermedias de comunicación entre procesos, aportan la flexibilidad en la definición de arquitectura de cada uno de ellos de manera separada, siempre que se mantenga una interfaz común
  - Escalabilidad: Con más unidades de procesamiento, las colas balancean su respectiva carga

© JMA 2020. All rights reserved

RabbitMQ

- RabbitMQ (https://www.rabbitmq.com/) es el intermediario de mensajes libre y de código abierto más ampliamente implementado (Broadcom ofrece suscripciones con soporte empresarial 24/7). Dicho de forma simple, es un software donde se pueden definir colas de mensajes, las aplicaciones se pueden conectar a dichas colas y transferir/leer mensajes en ellas.
- RabbitMQ es ligero y fácil de implementar en las instalaciones y en la nube. Es compatible con múltiples protocolos de mensajería: AMQP 0-9-1, AMQP 1.0, MQTT y STOMP. RabbitMQ se puede implementar en configuraciones distribuidas y federadas para cumplir con los requisitos de alta disponibilidad y alta escalabilidad.
- RabbitMQ esta disponible en muchos sistemas operativos y entornos de nube, y proporciona una amplia gama de herramientas para desarrolladores en los lenguajes más populares.
- La arquitectura básica de una cola de mensajes es simple. Hay aplicaciones clientes, llamadas productores, que crean mensajes y los entregan al intermediario (la cola de mensajes). Otras aplicaciones, llamadas consumidores, se conectan a la cola y se suscriben a los mensajes que se procesarán. Un mensaje puede incluir cualquier tipo de información.

© JMA 2020. All rights reserved

Conceptos

- Un cliente puede ser productor, consumidor o productor y consumidor de mensajes. Los mensajes colocados en la cola se almacenan hasta que el consumidor los recupera y procesa.
- Los mensajes no se publican directamente en una cola, en lugar de eso, el productor envía mensajes a un exchange. Los exchanges son agentes de enrutamiento de mensajes, definidos por virtual host dentro de RabbitMQ. Un exchange es responsable del enrutamiento de los mensajes a las diferentes colas: acepta mensajes del productor y los dirige a colas de mensajes con ayuda de atributos de cabeceras, bindings y routing keys.
  - Un binding es un «enlace» que se configura para vincular una cola o exchange a un exchange
  - La routing key es un atributo del mensaje. El exchange podría usar esta clave para decidir cómo enrutar el mensaje a las colas (según el tipo de exchange)
  - Un virtual host es una división lógica de los componentes de servidor de RabbitMQ en la cual estos se agrupan para simplificar su administración y control.
- Los exchanges, las conexiones y las colas pueden configurarse con parámetros tales como durable, temporary y auto delete en el momento de su creación. Los exchanges declarados como durable sobrevivirán a los reinicios del servidor y durarán hasta que se eliminen explícitamente. Aquellos de tipo temporary existen hasta que RabbitMQ se cierre. Por último, los exchanges configurados como auto delete se eliminan una vez que el último objeto vinculado se ha liberado del exchange.

© JMA 2020. All rights reserved

Colas

- Una cola en RabbitMQ es una colección ordenada de mensajes. Los mensajes se ponen en cola y se sacan de la cola (se entregan a los consumidores) siguiendo el método FIFO ("primero en entrar, primero en salir"). Una cola es una estructura de datos secuencial con dos operaciones principales: un elemento puede ponerse en cola (agregarse) en la cola y sacarse de la cola (consumirse) desde la cabeza.
- Las colas desempeñan un papel importante en el ámbito de la tecnología de mensajería. Muchos protocolos y herramientas de mensajería suponen que los productores y los consumidores se comunican mediante un mecanismo de almacenamiento similar a una cola.
- Algunas funciones de colas de RabbitMQ, como las prioridades y la puesta en cola de los consumidores, pueden afectar el orden que observan los consumidores.
- RabbitMQ da soporte a:
  - Colas (clásicas, tipo de cola original), es un tipo de cola versátil y de alto rendimiento adecuado para la mayoría de los casos de uso.
  - Cola de quórum (mitad+1), es un nuevo tipo de cola moderna que implementa replicación para cuando se necesita alta disponibilidad y donde la seguridad ante perdidas de datos es la máxima prioridad, aunque penalice el rendimiento.
  - Streams, es una estructura de datos replicada persistente de mensajería alternativa a las colas de mensajes .

© JMA 2020. All rights reserved

Colas

- Antes de poder utilizar una cola, es necesario declararla. Las aplicaciones pueden elegir nombres de colas o pedirle al agente que genere un nombre para ellas (colas anónimas).
- Las colas se pueden declarar como duraderas o transitorias. Los metadatos de una cola duradera se almacenan en el disco y la cola sobrevivirá a un reinicio del agente con los mensajes publicados como persistentes, mientras que los metadatos de una cola transitoria se almacenan en la memoria cuando es posible y las colas transitorias se eliminarán al iniciar el agente.
- En algunos casos de uso se supone que las colas tienen una vida útil breve, se pueden definir como de eliminación automática: la cola que ha tenido al menos un consumidor se elimina cuando el último consumidor cancela la suscripción. Alternativamente, se pueden definir como de exclusiva: solo puede ser utilizada (consumida, purgada, eliminada, …) por su conexión declarante y la cola se eliminará cuando se cierre esa conexión.
- Las colas pueden definir argumentos opcionales utilizados por complementos y funciones específicas del agente, como el TTL (tiempo de vida) para los mensajes y las colas, límite de longitud de la cola, límite de tamaño de la cola, comportamiento del desbordamiento de cola, …

© JMA 2020. All rights reserved

Exchanges

- RabbitMQ ofrece varios tipos de exchanges, cada uno con un comportamiento de enrutamiento diferente:
  - **Direct** Exchange: Enruta los mensajes a las colas que tienen una clave de enrutamiento (routing key) idéntica a la clave de enrutamiento del mensaje. Es útil cuando se necesita una correspondencia precisa entre las claves de enrutamiento.
  - **Topic** Exchange: Enruta los mensajes a una o mas colas basándose en patrones de enrutamiento definidos en las claves de enrutamiento. Los patrones pueden incluir comodines (* para una palabra, # para una o más palabras). Permite un enrutamiento más flexible en comparación con el Direct Exchange.
  - **Headers** Exchange: Enruta los mensajes a una o mas colas según los encabezados específicos del mensaje, en lugar de las claves de enrutamiento, que puede ser útil cuando los criterios de enrutamiento son más complejos y se basan en metadatos del mensaje. El atributo x-match en el enlace permite especificar: all (debe coincidir con todos los valores de encabezado) y any (debe coincidir con algún valor de encabezado)
  - **Fanout** Exchange: Enruta los mensajes a todas las colas que están vinculadas al exchange, sin importar las claves de enrutamiento. Es útil cuando se desea enviar el mismo mensaje a múltiples destinatarios.

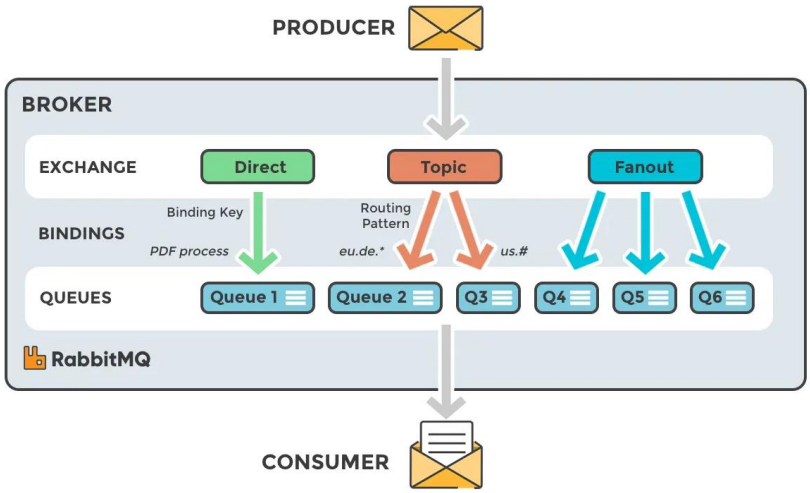
© JMA 2020. All rights reserved

Exchanges

- El enrutamiento esta basado en:
  - **Routing key**: es una etiqueta (la convención de nombres separada por puntos es muy común) asociada a un mensaje específico que se utiliza para determinar qué colas recibirán el mensaje. Los routing keys se basan en patrones de palabras clave y se utilizan en los intercambios de tipo topic para enviar mensajes a las colas que coinciden con el patrón definido en el mensaje.
  - **Binding key**: es la clave que se utiliza para vincular una cola específica a un intercambio de tipo topic. Cuando una cola está vinculada a un topic exchange con un binding key, solo recibirá los mensajes cuyo routing key coincida con el binding key definido.
- El default exchange es un direct exchange sin nombre (cadena vacía) pre-declarado por el agente. Tiene una propiedad especial que lo hace muy útil para aplicaciones simples: cada cola que se crea se enlaza con ella misma automáticamente mediante una clave de enrutamiento que es la misma que el nombre de la cola.
- A veces es conveniente dejar que los mensajes que un exchange no pudo enrutar (porque no había colas vinculadas o no había enlaces coincidentes) por un exchange genérico. El Alternate Exchange ("AE") es la función que aborda estos casos de uso. Para cualquier exchange, excepto el predeterminado, se puede definir un AE mediante políticas.

© JMA 2020. All rights reserved

Flujo



© JMA 2020. All rights reserved

Acuses de recibo

- Para garantizar que un mensaje nunca se pierda, RabbitMQ admite acuses de recibo (ACK o NACK) de mensajes. Existen dos modos principales de reconocimiento en RabbitMQ:
  - Modo Automático (Auto Acknowledgment): En este modo, una vez que RabbitMQ entrega un mensaje a un consumidor, lo considera entregado y lo quita de la cola inmediatamente, sin esperar un ACK. Esto significa que el consumidor no tiene que enviar un ACK manualmente. Sin embargo, esto también puede llevar a situaciones donde un mensaje se pierda si el consumidor falla antes de procesarlo.
  - Modo Manual (Manual Acknowledgment): En este modo, el consumidor debe enviar explícitamente un ACK al servidor una vez que ha procesado el mensaje y que puede eliminarlo. Esto permite un mayor control y garantiza que los mensajes no se pierdan fácilmente en caso de problemas en el consumidor. Con NACK le informa que debe reintegrarlo a la cola. Es un error común, con consecuencias graves, pasar por alto el ACK, dado que los mensajes se volverán a entregar cuando el consumidor se cierre (reintegra a la cola los mensajes no confirmados). Se puede configurar que si un consumidor no envía un ACK dentro de un cierto período de tiempo, RabbitMQ considere que el mensaje no ha sido entregado y lo vuelva a encolar.
- Se pueden configurar tiempos de vida para los mensajes para garantizar que los mensajes no queden encolados indefinidamente.
- Por defecto RabbitMQ enviará cada mensaje al siguiente consumidor, en secuencia (Round-Robin). Se puede configurar para que no entregue más de un mensaje a un consumidor a la vez: no enviara un nuevo mensaje a un consumidor hasta que haya procesado y reconocido el anterior.

© JMA 2020. All rights reserved

Conexiones y canales

- Los productores suelen tener una vida útil prolongada: es decir, a lo largo de la vida de un productor, este publica múltiples mensajes. Abrir una conexión o canal (sesión) para publicar un solo mensaje no es lo óptimo. A su vez, los consumidores están destinados a vivir mucho tiempo: es decir, a lo largo de su vida, un consumidor recibe múltiples entregas. Registrar a un consumidor para que reciba un solo mensaje no es lo óptimo.
- Los productores suelen abrir sus conexiones durante el inicio de la aplicación y los consumidores suelen registrarse durante el inicio de la aplicación. A menudo, las conexiones permanecen activas mientras se ejecuta la aplicación. Los publicadores pueden ser más dinámicos y comenzar a publicar en respuesta a un evento del sistema, deteniéndose cuando ya no son necesarios, y los consumidores también pueden ser más dinámicos registrándose como reacción a un evento del sistema y cancelando la suscripción cuando ya no son necesarios. Esto es común con los clientes WebSocket que se utilizan a través de los complementos Web STOMP y Web MQTT, los clientes móviles, etc.
- Algunas aplicaciones necesitan varias conexiones lógicas al agente. Sin embargo, no es conveniente mantener abiertas muchas conexiones TCP al mismo tiempo, ya que esto consume recursos del sistema y dificulta la configuración de los cortafuegos. Las conexiones AMQP 0-9-1 se multiplexan y los canales (abstracción específica de AMQP 0-9-1) pueden considerarse como "conexiones livianas que comparten una única conexión TCP". Cada operación de protocolo que realiza un cliente ocurre en un canal. La comunicación en un canal en particular es completamente independiente de la comunicación en otro canal, por lo tanto, cada método de protocolo también lleva un ID de canal (número entero de canal), que tanto el agente como los clientes usan para determinar a qué canal va dirigido el método. Un canal sólo existe en el contexto de una conexión y nunca por sí solo. Cuando se cierra una conexión, también se cierran todos los canales que la componen.

© JMA 2020. All rights reserved

Carga útil

- RabbitMQ es un sistema de mensajería que maneja mensajes en forma de datos serializados y no impone restricciones específicas sobre los tipos de datos que pueden ser enviados en los mensajes, ya que los mensajes son simplemente secuencias de bytes. Algunos ejemplos de formatos de datos que se pueden enviar son:
  - Texto: Cadenas de texto en mensajes para transmitir información legible por humanos.
  - JSON o XML: Es común enviar datos estructurados en formato JSON o XML para intercambiar información entre sistemas.
  - Datos Binarios: Se pueden enviar archivos y datos binarios (imágenes, videos, documentos, etc.) codificados en base64 u otros formatos en los mensajes.
  - Objetos Serializados: En lenguajes que soportan serialización de objetos (como Java o C# con objetos serializables), se pueden enviar objetos serializados como mensajes.
  - Formatos Propios: Definiendo cómo los datos serán codificados y decodificados por los productores y consumidores.
- Es importante tener en cuenta que tanto el productor como el consumidor deben estar de acuerdo en el formato de los datos que se están intercambiando.
- Es recomendable seleccionar un formato que sea eficiente para el uso de los recursos y que permita una fácil interpretación por parte de los sistemas que envían y reciben los mensajes.

© JMA 2020. All rights reserved

Instalación

- Descargar la última versión estable para Windows desde:
  - https://www.rabbitmq.com/download.html
- Para arrancar el servicio:
  - rabbitmqctl.bat start
- Para parar el servicio:
  - rabbitmqctl.bat stop
- Instalación del complemento de administración:
  - rabbitmq-plugins enable rabbitmq_management
- Para crear un contenedor:
  - docker run -d --name rabbitmq -p 4369:4369 -p 5671:5671 -p 5672:5672 -p 15691:15691 -p 15692:15692 -p 25672:25672 -p 15671:15671 -p 15672:15672 -e RABBITMQ_DEFAULT_USER=admin -e RABBITMQ_DEFAULT_PASS=curso rabbitmq:management-alpine
- Para acceder al complemento de administración:
  - http://localhost:15672
  - El agente crea un usuario “guest” con contraseña “guest”.

© JMA 2020. All rights reserved

Port Access

- 4369: epmd , un servicio de descubrimiento de pares utilizado por los nodos RabbitMQ y las herramientas CLI
- 5672, 5671: utilizado por clientes AMQP 0-9-1 y AMQP 1.0 con y sin TLS
- 5552, 5551: utilizado por los clientes del protocolo RabbitMQ Stream sin y con TLS
- 6000 a 6500: se utilizan para stream replication
- 25672: se utiliza para la comunicación entre nodos y herramientas CLI (puerto del servidor de distribución Erlang) y se asigna desde un rango dinámico (limitado a un solo puerto de manera predeterminada, calculado como puerto AMQP + 20000).
- 35672-35682: este rango de puertos TCP del cliente lo utilizan las herramientas CLI para comunicarse con los nodos . De manera predeterminada, el rango calculado es (server distribution port + 10000)a través de(server distribution port + 10010)
- 15672, 15671: Clientes API HTTP , interfaz de administración y rabbitmqadmin , sin y con TLS (solo si el complemento de administración está habilitado)
- 61613, 61614: Clientes STOMP sin y con TLS (solo si el complemento STOMP está habilitado)
- 1883, 8883: Clientes MQTT sin y con TLS, si el complemento MQTT está habilitado
- 15674: Clientes STOMP sobre WebSockets (solo si el complemento Web STOMP está habilitado)
- 15675: Clientes MQTT sobre WebSockets (solo si el complemento Web MQTT está habilitado)
- 15692, 15691: Métricas de Prometheus, sin y con TLS (solo si el complemento Prometheus está habilitado)

© JMA 2020. All rights reserved

RabbitMQ: Spring Boot

- Añadir al proyecto:
  - Messaging > Spring for RabbitMQ
- Configurar:

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
#spring.rabbitmq.template.retry.enabled=true
#spring.rabbitmq.template.retry.initial-interval=2s
```

- AmqpTemplate y AmqpAdmin se configuran de forma automáticamente.
- Anotaciones:
  - @RabbitListener: indica que es un método de escucha de una cola.
  - @Queue: definición de una cola.
  - @Exchange: definición de un enrutador
  - @QueueBinding: Define una cola, el intercambio al que debe vincularse y una clave de enlace opcional, utilizada con @RabbitListener.

© JMA 2020. All rights reserved

Declaraciones

- Crear cola si no existe:

```
@Bean
Queue myQueue(@Value("${app.cola}") String queue) {
   return new Queue(queue);
}
```

- Crear un exchange si no existe:

```
@Bean
FanoutExchange myExchange(@Value("${app. exchange}") String exchange) {
        return new FanoutExchange(exchange);
}
```

- Vincular una cola a un exchange:

```
@Bean
Binding myBinding(FanoutExchange myExchange, Queue myQueue) {
        return BindingBuilder.bind(myQueue).to(myExchange);
}
```

© JMA 2020. All rights reserved

Manejar una cola

- Enviar un mensaje:

```
@Autowired
private AmqpTemplate amqp;

public void send(String cola, MessageDTO outMsg) {
    amqp.convertAndSend(cola, outMsg);
}
```

- Solicitar el siguiente mensaje (imperativo):

```
public void getNextMessage() {
    var next = amqp.receiveAndConvert(myQueue.getName());
    if(last == null) // cola vacia, no había mensaje
```

- Recibir mensajes (declarativo):

```
@RabbitListener(queues = "mi-cola")
public void recive(MessageDTO inMsg) {
    // Procesar el mensaje recibido: inMsg
}
```

© JMA 2020. All rights reserved

Formato del Mensaje

- En la configuración, crear el formateador de los mensajes:

```
@Bean
public MessageConverter jsonConverter() () {
        return new Jackson2JsonMessageConverter();
}
```

- En la configuración, crear la versión personalizada del RabbitTemplate con el formateador recién creado:

```
@Bean
public RabbitTemplate rabbitTemplate(final ConnectionFactory connectionFactory) {
    RabbitTemplate rabbitTemplate = new RabbitTemplate(connectionFactory);
    rabbitTemplate.setMessageConverter(jsonConverter());
    return rabbitTemplate;
}
```

- Aplicar el formateador de los mensajes solo a determinados receptores de mensajes:

```
@RabbitListener(queues = "mi-cola", messageConverter = "jsonConverter")
public void recive(MessageDTO inMsg) {
```

© JMA 2020. All rights reserved

@RabbitListener

- La siguiente lista muestra los argumentos que están disponibles para coincidir con los parámetros en los métodos anotados con @RabbitListener:
  - org.springframework.amqp.core.Message sin procesar.
  - MessageProperties del Message sin procesar.
  - El com.rabbitmq.client.Channel sobre el cual se recibió el mensaje.
  - El org.springframework.messaging.Message convertido del mensaje AMQP entrante.
  - @Header para extraer un valor de encabezado específico, incluidos los encabezados AMQP estándar y @Headers, asignable a java.util.Map, para obtener acceso a todos los encabezados.
  - Un parámetro no anotado que no es uno de los anteriores se corresponde con la carga útil. Puede hacerse explícitamente anotando el parámetro con @Payload.
- @RabbitListener puede declarar lo que necesite:

```
@RabbitListener(bindings = @QueueBinding(value = @Queue(value = "myQueue", durable = "true"),
    exchange = @Exchange(value = "auto.exch", ignoreDeclarationExceptions = "true"),
    key = "orderRoutingKey")
)
public void processOrder(Order order, @Header("order_type") String orderType) {
```

© JMA 2020. All rights reserved

Acuse de recibo del mensaje

- Cuando un consumidor recibe un mensaje, el procesamiento de ese mensaje puede o no tener éxito. Al rechazar un mensaje, el consumidor indica al intermediario que el procesamiento del mensaje ha fallado (o que no se puede realizar en ese momento) y le puede pedir que lo descarte o lo reintegre a la cola (pierde su posición). Se puede configurar un Exchange para que los mensajes descartados acaben en una cola de mensajes fallidos (dead letter).
- El acuse de recibo debe enviarse por el mismo canal en el que se recibió la entrega.

```
channel.basicAck();
channel.basicReject(deliveryTag, requeue);
```

- Si el método anotado con @RabbitListener termina sin excepciones envía un Ack. Si lanza una AmqpRejectAndDontRequeueException, se rechaza el mensaje y se descarta. Si lanza una AmqpException o ImmediateRequeueAmqpException, se rechaza el mensaje y se reintegra a la cola.

© JMA 2020. All rights reserved

Gestión de respuestas

- RPC (solicitud/respuesta) es un patrón popular para implementar con un agente de mensajería como RabbitMQ. Un cliente envía un mensaje de solicitud y un servidor responde con un mensaje de respuesta. Para recibir una respuesta, necesitamos enviar una dirección de cola de "devolución de llamada" con la solicitud.
- El protocolo AMQP 0-9-1 predefine un conjunto de 14 propiedades que acompañan a un mensaje. La mayoría de las propiedades se utilizan con poca frecuencia, con excepción de las siguientes:
  - deliveryMode: Marca un mensaje como persistente (con un valor de 2) o transitorio (cualquier otro valor). Es posible que recuerdes esta propiedad del segundo tutorial .
  - contentType: Se utiliza para describir el tipo MIME de la codificación. Por ejemplo, para la codificación JSON que se utiliza con frecuencia, es una buena práctica establecer esta propiedad en: application/json.
  - replyTo: Se utiliza comúnmente para nombrar una cola de devolución de llamada.
  - correlationId: Útil para correlacionar las respuestas de RPC con las solicitudes.
- Spring AMQP permite centrarse en el estilo de mensaje con el que está trabajando y ocultar los detalles de la interconexión de mensajes necesarios para admitir este estilo.

```
asyncRabbitTemplate.convertSendAndReceive(rpcExchange.getName(), routingKey, peticion)
    .thenAccept(result -> procesaRespuesta(result))
    .exceptionally(ex -> { LOGGER.severe(ex.toString()); return null; });
```

© JMA 2020. All rights reserved

Gestión de respuestas

- El soporte existente de @RabbitListener permite que el método tenga un tipo de retorno distinto de void. Cuando ese es el caso, el resultado de la invocación se encapsula en un mensaje enviado a la dirección especificada en el encabezado ReplyToAddress del mensaje original o a la dirección predeterminada configurada en el receptor.

```
@RabbitListener(destination = "myQueue")
public OrderStatus processOrder(Order order) {
    // order processing
    return status;
}
```

- Puede configurar esa dirección predeterminada mediante la anotación @SendTo:

```
@RabbitListener(destination = "myQueue")
@SendTo("resultQueue")
public Message<OrderStatus> processOrder(Order order) {
    // order processing
    return MessageBuilder.withPayload(status).setHeader("code", 1234).build();
}
```

© JMA 2020. All rights reserved

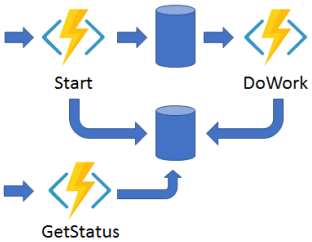
WebHooks

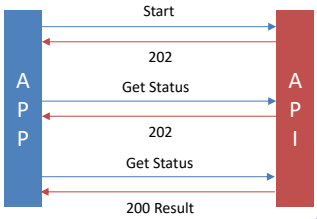
- Los webhooks son eventos HTTP que desencadenan acciones. Su nombre se debe a que funcionan como «enganches» de los programas en Internet y casi siempre se utilizan para la comunicación entre sistemas. Son la manera más sencilla de obtener un aviso cuando algo ocurre en otro sistema y para el intercambio de datos entre aplicaciones web, permitiendo las llamada asíncronas en HTTP.
- Un webhook es una retro llamada HTTP, una solicitud HTTP GET/POST insertada en una página web, que interviene cuando ocurre algo (una notificación de evento a través de HTTP GET/POST).
- Los webhooks se utilizan para las notificaciones en tiempo real (con los datos del evento como parámetros o en cuerpo en JSON o XML) a una determinada dirección http:// o https://, que puede:
  - almacenar los datos del evento en JSON o XML
  - generar una respuesta que permita actualizarse al sistema donde se produce el evento
  - ejecutar un proceso en el sistema receptor del evento (Ej: enviar un correo electrónico)
- Los webhook están pensados para su utilización desde páginas web y sus diferentes consumidores: navegadores, correo electrónico, webapps, …
- Un ejemplo típico es su utilización en correos electrónico de marketing para notificar al servidor que debe enviar un nuevo correo electrónico porque el usuario a abierto el mensaje.
- Pueden considerarse una versión especializada y simplificada de los servicios REST (solo GET/POST).

© JMA 2020. All rights reserved

WebHooks

- El patrón Async HTTP APIs soluciona el problema de coordinar el estado de las operaciones de larga duración con los clientes externos. Una forma habitual de implementar este patrón es que un punto de conexión HTTP desencadene la acción de larga duración. A continuación, el cliente se redirige a un punto de conexión de estado al que sondea para saber el estado actual del proceso y cuando finaliza la operación.
- Un endpoint HTTP desencadena el proceso con la acción de larga duración y devuelve inmediatamente un 202 si la petición es correcta y, opcionalmente, como carga útil las URLs de los endpoints de estado y control. Las peticiones incorrectas recibirán el 4XX apropiado.
- El cliente sondea periódicamente el endpoint de estado y obtiene:
  - 202 con el estado actual del proceso como carga útil mientras el proceso este en marcha.
  - 200 con el estado final (correcto o fallido) del proceso como carga útil cuando haya finalizado la operación.
- El endpoint HTTP desencadenador puede suministrar endpoints adicionales para pausar, reanudar, cancelar/terminar, reiniciar o enviar eventos al proceso en marcha.

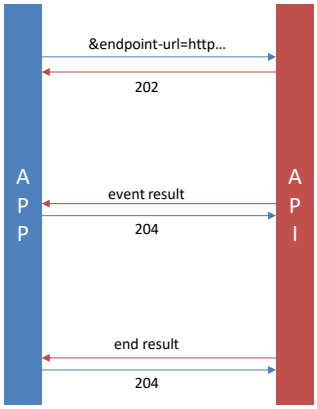




© JMA 2020. All rights reserved

WebHooks

- El patrón Reverse API soluciona el problema del sondeo periódico, las API inversas invierten esta situación, para que sea la API invocada la que notifique automáticamente cuando se ha producido un determinado evento. El cliente debe crear su propia API para recibir las notificaciones.
- Al realizar la petición al endpoint HTTP desencadenador, se suministra un endpoint propio para que el proceso desencadenado pueda notificar cuándo ocurre algo de interés. Le da la vuelta a la comunicación, el cliente pasa a ser servidor y el servidor a cliente. El desencadenador devuelve inmediatamente un 202 si la petición es correcta o el 4XX apropiado se es incorrecta.
- Este esquema de funcionamiento tiene muchas ventajas en ambos extremos:
  - **Ahorro de recursos y tiempo**: con el sondeo se harán muchas llamadas "para nada", que no devolverán información relevante. Los dos extremos gastan recursos para hacer y responder a muchas llamadas que no tienen utilidad alguna (no nos llame, ya les llamaremos).
  - **Eliminación de los retrasos**: la aplicación usuaria recibirá una llamada en el momento exacto en el que se produce y no tendrá retrasos al próximo sondeo.
  - **Velocidad de las llamadas**: generalmente la llamada que se hace a un webhook es muy rápida porque solo se envía una pequeña información sobre el evento y se suele procesar asíncronamente. Muchas veces ni siquiera se espera por el resultado: se hace una llamada del tipo "fire and forget" (o sea, dispara y olvídate), pues se trata de notificar el evento y listo.



© JMA 2020. All rights reserved

APACHE KAFKA

© JMA 2020. All rights reserved

Eventos

- Un evento es una notificación que indica un cambio en un entorno, una aplicación o un servicio.
- En una arquitectura basada en eventos, los productores de eventos generan un flujo de eventos que los consumidores de eventos escuchan. Los productores están desconectados entre sí y de los consumidores, no saben si los consumidores están escuchando. A diferencia de los mensajes, un evento puede ser consumido por múltiples consumidores o por ninguno. Dado que un evento puede no ser consumido, la carga útil suele estar reducida al máximo en comparación con los mensajes.
- La transmisión de eventos (event streaming) es la práctica de:
  - capturar datos en tiempo real de las fuentes de eventos (como bases de datos, sensores, dispositivos móviles, IoT, servicios en la nube y aplicaciones de software) en forma de flujos de eventos
  - almacenar estos flujos de eventos de forma duradera para su posterior recuperación
  - manipular, procesar y reaccionar a los flujos de eventos en tiempo real y retrospectivamente
  - enrutar los flujos de eventos a diferentes tecnologías de destino según sea necesario.
- La transmisión de eventos garantiza un flujo continuo y una interpretación de los datos para que la información correcta esté en el lugar correcto y en el momento correcto.
- Los eventos se entregan casi en tiempo real, de modo que los consumidores pueden responder inmediatamente a los eventos cuando se producen. Un consumidor puede unirse en cualquier momento y capturar tantos eventos anteriores como desee.

© JMA 2020. All rights reserved

Casos de uso

- **Procesamiento de flujos**: procesar datos en canales de procesamiento que constan de múltiples etapas, donde los datos de entrada sin procesar se consumen de temas y luego se agregan, enriquecen o transforman de otro modo en nuevos temas para su posterior consumo o procesamiento de seguimiento.
- Métricas y **Telemetría**: se utiliza a menudo para datos de seguimiento operativo. Esto implica agregar estadísticas de aplicaciones distribuidas para producir fuentes centralizadas de datos operativos.
- **Log Aggregation** (Agregación de registros): como sustituto de una solución de agregación de registros. La agregación de registros generalmente recopila archivos de registro físicos de los servidores y los coloca en un lugar central (quizás un servidor de archivos o HDFS) para su procesamiento.
- **Event Sourcing**: El seguimiento de eventos es un estilo de diseño de aplicaciones en el que los cambios de estado se registran como una secuencia de registros ordenada en el tiempo.
- **Seguimiento** de actividad del sitio web: registrar toda la actividad del usuario en sitios web (requiere cookies). El seguimiento de la actividad suele tener un volumen muy alto, ya que se generan muchos mensajes de actividad para cada vista de página de usuario.
- Registro de confirmación (**No repudio**): puede servir como una especie de registro de confirmación externo para un sistema distribuido.

© JMA 2020. All rights reserved

Apache Kafka

- Apache Kafka es una popular plataforma de streaming de eventos y de código abierto, que sirve para recoger, procesar y almacenar datos de eventos de streaming o datos sin principio ni final concretos. Kafka posibilita aplicaciones distribuidas que pueden escalar para gestionar miles de millones de eventos de streaming cada minuto y casi en tiempo real.
- Al estar diseñado específicamente para la transmisión de registros en tiempo real, Apache Kafka es ideal para las aplicaciones que requieren:
  - Intercambio de datos fiables entre diferentes componentes
  - La capacidad de dividir las cargas de trabajo de mensajería a medida que cambian los requisitos de la aplicación
  - Transmisión en tiempo real para el procesamiento de datos
  - Soporte nativo para la reproducción de datos/mensajes

© JMA 2020. All rights reserved

Instalación

- Hay varias opciones disponibles para instalar y configurar el entorno. Se pueden instalar los componentes necesarios en la máquina host de forma tradicional, pero es mas conveniente utilizar contenedores Docker dado que tiene la ventaja de ser limpia, independiente del sistema y fácil de configurar.
- Los componentes imprescindibles para operar con Apache Kafka son:
  - Kafka Brokers: Cada uno de los servicios intermediarios de Kafka que conforman el clúster. Almacenan y distribuyen los datos, que se organizan en topics.
  - Zookeeper: Gestiona los brokers de Kafka y les envía notificaciones en caso de cambio como creación de topics, borrado de topics, caída de broker, recuperación de broker …
  - UI for Apache Kafka (opcional): Es una interfaz web de usuario gratuita y de código abierto para monitorizar y administrar clústeres de Apache Kafka.

© JMA 2020. All rights reserved

Docker Compose

```
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:latest
    container_name: zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - 2181:2181

  kafka:
    image: confluentinc/cp-kafka:latest
    container_name: kafka
    depends_on:
      - zookeeper
    ports:
      - 9092:9092
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  kafka-ui:
    image: provectuslabs/kafka-ui
    container_name: kafka-ui
    depends_on:
      - kafka
    ports:
      - 9091:8080
    environment:
      - KAFKA_CLUSTERS_0_NAME=local
      - KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:29092
      - KAFKA_CLUSTERS_0_ZOOKEEPER=localhost:2181
```

© JMA 2020. All rights reserved

Conceptos

- Un **evento** registra el hecho de que "algo sucedió" en el mundo o en el negocio. También se le llama registro o mensaje en la documentación. Cuando se leen o escriben datos en Kafka, se hace en forma de eventos. Conceptualmente, un evento tiene una clave, un valor, una marca de tiempo y encabezados de metadatos opcionales.
- Los **productores** son aquellas aplicaciones cliente que publican (escriben) eventos en Kafka, y los **consumidores** son los que se suscriben (leen y procesan) estos eventos. Los consumidores se pueden agrupar para que solo un miembro del grupo reciba y procese el evento. En Kafka, los productores y los consumidores están totalmente desvinculados y son independientes entre sí, lo cual es un elemento de diseño clave para lograr la alta escalabilidad por la que Kafka es conocido.
- Los eventos se organizan y almacenan de forma duradera en temas con nombre (**topics**), similar a las colas de mensajes. Los temas en Kafka son siempre multiproductor y multisuscriptor: un tema puede tener cero, uno o muchos productores que escriben eventos en él, así como cero, uno o muchos consumidores que se suscriben a estos eventos. Los eventos de un tema se pueden leer tantas veces como sea necesario; a diferencia de los sistemas de mensajería tradicionales, los eventos no se eliminan después del consumo. En su lugar, se define a través de la configuración del tema durante cuánto tiempo Kafka debe retener los eventos, después del cual se descartarán los eventos antiguos. El rendimiento de Kafka es efectivamente constante con respecto al tamaño de los datos, por lo que almacenar datos durante mucho tiempo puede estar perfectamente bien.

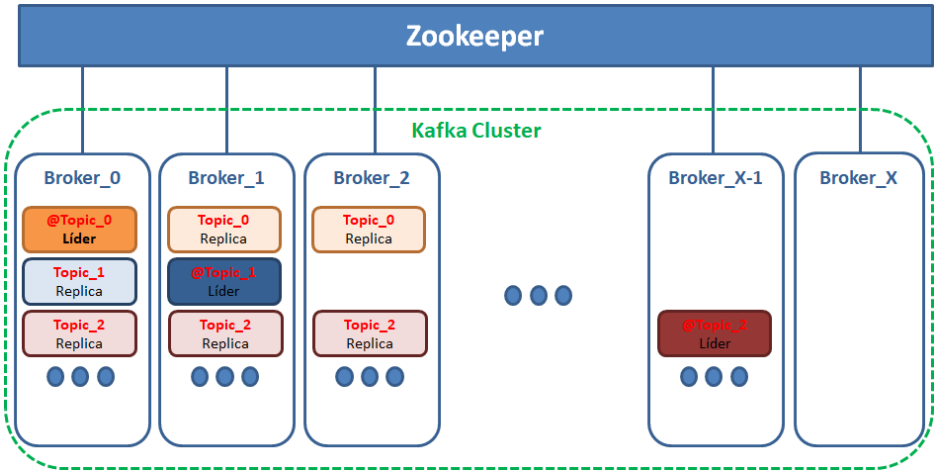
© JMA 2020. All rights reserved

Conceptos

- Los temas están **particionados**, lo que significa que un tema se distribuye en una o varias "agrupaciones" ubicadas en diferentes intermediarios (brokers) de Kafka. Esta ubicación distribuida de los datos es muy importante para la escalabilidad porque permite que las aplicaciones de los clientes lean y escriban los datos desde/hacia muchos intermediarios al mismo tiempo. Cuando se publica un nuevo evento en un tema, en realidad se agrega a una de las particiones del tema. Los eventos con la misma clave de evento se escriben en la misma partición, y Kafka garantiza que cualquier consumidor de una partición de un tema determinado siempre leerá los eventos de esa partición exactamente en el mismo orden en que se escribieron. A cada mensaje se le asigna un número incremental u **offset** que identifica cada mensaje dentro de la partición, la diferencia entre el offset producido y el offset consumido se denomina **lag**, con una valor cercano a cero significa que el consumidor está procesando los mensajes sin retardo y con valores altos y que se incrementan significa que el consumidor no está siendo capaz de procesar el volumen de mensajes producidos.
- Para que los datos sean tolerantes a fallos y estén altamente disponibles, cada tema se puede **replicar**, incluso entre regiones geográficas o centros de datos, de modo que siempre haya varios intermediarios que tengan una copia de los datos en caso de que las cosas salgan mal, para hacer el mantenimiento del intermediario o cualquier otra contingencia. Una configuración de producción común es un factor de replicación de 3, es decir, siempre habrá tres copias de los datos. Esta replicación se realiza a nivel de particiones de tema.

© JMA 2020. All rights reserved

Conceptos



© JMA 2020. All rights reserved

Conceptos

- Los **grupos de consumidores**, identificados por un Group ID común, permiten la escalabilidad horizontal y el paralelismo en el procesamiento de datos. Cada partición de un tópico solo puede ser consumida por un consumidor dentro de un grupo en un momento dado, si se desconecta salta al siguiente. Esto permite que múltiples consumidores dentro de un grupo trabajen en paralelo y procesen los datos de manera eficiente.
- Apache Kafka tiene un modelo de entrega de mensajes garantizado llamado "**acknowledgment**" (reconocimiento), denominado "ack", que permite a los productores saber si un mensaje ha sido entregado correctamente a los brokers y, por lo tanto, está disponible para ser consumido por los consumidores. Kafka garantiza que el mensaje ha sido entregado según el nivel de ack especificado pero no garantiza que el mensaje haya sido consumido por un consumidor: la confirmación de entrega se refiere a la disponibilidad del mensaje en los brokers para su consumo. Para verificar si un mensaje ha sido consumido, los consumidores pueden confirmar su procesamiento por separado y luego avanzar el offset de la partición para indicar que han leído y manejado ese mensaje.
- Kafka no impone ninguna restricción en el **contenido** de los mensajes, los trata simplemente como una cantidad de bytes ya sean de texto, comprimidos o cifrados.

© JMA 2020. All rights reserved

API de Kafka

- **Admin API**: para administrar e inspeccionar temas, intermediarios y otros objetos de Kafka.
- **Producer API**: para publicar (escribir) un flujo de eventos en uno o más temas de Kafka.
- **Consumer API**: para suscribirse a (leer) uno o más temas y procesar el flujo de eventos producidos para ellos.
- **Kafka Streams API**: para implementar microservicios y aplicaciones de procesamiento de flujo. Proporciona funciones de nivel superior para procesar flujos de eventos, incluidas transformaciones, operaciones con estado como agregaciones y uniones, ventanas, procesamiento basado en el tiempo del evento y más. La entrada se lee de uno o más temas para generar una salida a uno o más temas, transformando efectivamente los flujos de entrada en flujos de salida.
- **Kafka Connect API**: para crear y ejecutar conectores de importación/exportación de datos reutilizables que consumen (leen) o producen (escriben) flujos de eventos desde y hacia sistemas y aplicaciones externos para que puedan integrarse con Kafka, hay cientos de conectores listos para usar.
- Herramientas de línea de comandos para tareas de gestión y administración.

© JMA 2020. All rights reserved

Apache Kafka: Spring Boot

- Añadir al proyecto:
  - Messaging > Spring for Apache Kafka
- Configurar:

```
spring.kafka.bootstrap-servers=${KAFKA_BROKER:http://localhost:9092}
```

- KafkaTemplate<String, String> se configuran de forma automáticamente.

```
@Autowired
private KafkaTemplate<String, String> kafkaTemplate;
```

- Para crear un tema si no existe:

```
@Bean
public NewTopic topicLocation() {
    return new NewTopic("mi-tema", 1, (short) 1);
}
```

© JMA 2020. All rights reserved

Enviar un evento

```
// la clave (key) es opcional
public void sendMessage(String topic, String key, String message) {
        kafkaTemplate.send(topic, key, message)
            .thenAccept(result -> LOG.info(String.format("TOPIC: %s, KEY: %s,
                MESSAGE: %s, OFFSET: %s", topic, key, message,
                result.getRecordMetadata().offset())))
            .exceptionally(ex -> {
                LOG.error(String.format("TOPIC: %s, KEY: %s, MESSAGE: %s, ERROR:
                    %s", topic, key, message, ex.getMessage()));
                return null;
            });
}
```

© JMA 2020. All rights reserved

Recibir eventos

- Recibir los mensajes de los eventos:

```
@KafkaListener(topics = "${topic.name}", groupId = "nombre-consumidor")
public void listenTopic(String message) {
    LOG.info("Mensaje: " + message);
}
```

- Extraer los metadatos de los eventos:

```
@KafkaListener(topics = "${topic.name}", topicPattern = "${topic.name}", groupId = "nombre-
    consumidor")
public void listenWithHeaders(@Header(KafkaHeaders.RECEIVED_KEY) String key, @Payload String
    message, @Header(KafkaHeaders.OFFSET) String offset) {
    LOG.info(String.format("KEY: %s, MESSAGE: %s, OFFSET: %s", key, message, offset));
}
```

- Para recibir todos los eventos anteriores del tema:
  - spring.kafka.consumer.auto-offset-reset=earliest

© JMA 2020. All rights reserved

Apache Kafka: .NET

- Añadir al proyecto:
  - dotnet add package Confluent.Kafka
- Para crear un tema si no existe:

```
using Confluent.Kafka;
using Confluent.Kafka.Admin;

using(var adminClient = new AdminClientBuilder(new AdminClientConfig { BootstrapServers = brokerList
}).Build()) {
   try {
      await adminClient.CreateTopicsAsync([
         new TopicSpecification { Name = topicName, ReplicationFactor = 1, NumPartitions = 1 }
      ]);
   } catch(CreateTopicsException e) {
      if(!e.Results[0].Error.Reason.Contains("already exists"))
         Console.WriteLine($"An error occurred creating topic {e.Results[0].Topic}: {e.Results[0].Error.Reason}");
   }
}
```

© JMA 2020. All rights reserved

Enviar un evento

```
// la clave (key) es opcional
var config = new ProducerConfig { BootstrapServers = brokerList };
using(var producer = new ProducerBuilder<string, string>(config).Build()) {
   for(int i = 1; i <= 100; i++) {
      try {
         var delay = Random.Shared.Next(500, 1500);
         Thread.Sleep(delay);
         var message =new Message<string, string> { Key = "Emisor-01", Value = $"Evento {i}"};
         var deliveryReport = await producer.ProduceAsync(topicName, message);
         Console.WriteLine($"entrega ({delay / 1000.0}s): {deliveryReport.Offset} {message.Value}");
      } catch(ProduceException<string, string> e) {
         Console.WriteLine($"failed to deliver message: {e.Message} [{e.Error.Code}]");
      }
   }
}
```

© JMA 2020. All rights reserved

Recibir eventos

```
CancellationTokenSource cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true;  cts.Cancel(); };
var cancellationToken = cts.Token;
var config = new ConsumerConfig { BootstrapServers = brokerList, GroupId = groupName, EnableAutoOffsetStore = false, EnableAutoCommit = true };
using(var consumer = new ConsumerBuilder<string, string >(config).SetErrorHandler((_, e) => Console.WriteLine($"Error: {e.Reason}")).Build()) {
   consumer.Subscribe(topicName);
   try {
      while(true) {
         try {
            var consumeResult = consumer.Consume(cancellationToken);
            if(consumeResult.IsPartitionEOF) {
               Console.WriteLine($"Reached end of topic {consumeResult.Topic}, partition {consumeResult.Partition}, offset {consumeResult.Offset}.");
               continue;
            }
            Console.WriteLine($"Received {consumeResult.Offset}: {consumeResult.Message.Key} - {consumeResult.Message.Value}");
            try { consumer.StoreOffset(consumeResult); } catch(KafkaException e) { Console.WriteLine($"Store Offset error: {e.Error.Reason}"); }
         } catch(ConsumeException e) {  Console.WriteLine($"Consume error: {e.Error.Reason}"); }
      }
   } catch(OperationCanceledException) {
      Console.WriteLine("Closing consumer.");
      consumer.Close();
   }
}
```

© JMA 2020. All rights reserved

ACTIVEMQ (JMS)

© JMA 2020. All rights reserved

ActiveMQ (JMS)

- Apache ActiveMQ es un intermediario de mensajes basado en Java, multiprotocolo y de código abierto, que implementa plenamente la especificación de Java Message Service 1.1 (JMS).
- Es compatible con los protocolos estándar de la industria (AMQP, STOMP sobre websockets, MQTT y JMS) para que los usuarios obtengan los beneficios de las opciones del cliente en una amplia gama de lenguajes y plataformas (JavaScript, C, C++, Python, .Net y más).
- Actualmente hay dos "sabores" de ActiveMQ disponibles: el conocido como broker "clásico" y el broker de "próxima generación" cuyo nombre en código es Artemis. Una vez que Artemis alcance un nivel suficiente de paridad de funciones con el broker "clásico", se convertirá en la versión principal de ActiveMQ.
- El broker "Classic" ofrece características empresariales tales como JMS 1.1 con implementación de cliente completa que incluye JNDI, alta disponibilidad mediante almacenamiento compartido, modelo familiar de direccionamiento basado en JMS, "Red de intermediarios" para la distribución de carga y KahaDB o JDBC para persistencia.
- Artemis ofrecerá JMS 1.1 y 2.0 + Jakarta Messaging 2.0 y 3.0 con implementaciones de cliente completas (incluido JNDI), alta disponibilidad mediante almacenamiento compartido o replicación de red, modelo simple y potente de direccionamiento agnóstico de protocolo, agrupación flexible para distribuir la carga, implementaciones avanzadas del diario para persistencia de baja latencia (así como JDBC), duplicación asíncrona para recuperación ante desastres y equilibrio de carga basado en datos.
- ActiveMQ "Artemis" dispondrá de una alta paridad de características con ActiveMQ "Classic" para facilitar la migración.

© JMA 2020. All rights reserved

Instalación

- Descargar la última versión estable para Windows desde:
  - https://activemq.apache.org/components/classic/download/
- Extraer los archivos del archivo ZIP en el directorio de instalación.
- Para arrancar el servicio, desde una ventana de consola, cambiar al directorio de instalación y ejecutar:
  - bin\activemq start
- Los directorios de trabajo se crean en relación con el directorio actual. Para crear directorios de trabajo en el lugar adecuado, ActiveMQ debe iniciarse desde su directorio de inicio/instalación.
- Para acceder al complemento de administración, desde el navegador:
  - URL: http://127.0.0.1:8161/admin/
    - Iniciar sesión: admin
    - Contraseña: admin
  - Ir a "Queues"
  - Agregar un nombre de cola y hacer clic en crear
  - Enviar mensaje de prueba haciendo clic en "Enviar a"
  - Consultar con atom o rss

© JMA 2020. All rights reserved

Protocolos

- Apache ActiveMQ es un intermediario de mensajes que admite múltiples protocolos de nivel de conexión para una máxima interoperabilidad.
  - Core-UI (port: 8161)
  - REST (port: 8161)
  - RSS y Atom (port: 8161)
  - AMQP (port: 5672)
  - MQTT (port: 1883)
  - WebSocket (port: 61614)
  - Stomp (port: 61613)
  - JMS (port: 61616)
  - OpenWire (port: 61616)
  - ~~XMPP (port: 61222)~~

© JMA 2020. All rights reserved

Conceptos

- Soporta dos modelos/dominios de mensajería:
  - Punto a Punto (Queues), en la que un mensaje se consume por un único consumidor.
  - Publicación/Subscripción (Topics), en la que un mensaje se consume por muchos consumidores.
- Soporta el modelo de mensaje unificado de JMS que especifica que todos los mensajes deben contar con cabeceras, propiedades y cuerpo. Las cabeceras ofrecen metadatos sobre el mensaje utilizado por ambos clientes y el proveedor JMS. Las propiedades son campos opcionales dentro del mensaje para añadir información adicional al mensaje. El cuerpo puede contener tanto texto como datos binarios mediante los diferentes tipos de mensajes.
- JMS define seis tipos de cuerpo para los mensajes, también conocidos tipos de mensajes:
  - Message: Tipo de mensaje base. Se utiliza para enviar un mensaje sin cuerpo, solo cabeceras y propiedades. Normalmente se utiliza para notificación simple de eventos.
  - MapMessage: Compuesto de un conjunto de pares {nombre,valor}. El tipo de los nombres es String, y los valores tipos primitivos Java. A los nombres (que no están ordenados) podemos acceder de forma secuencial mediante un enumerador, o por acceso directo por nombre.
  - BytesMessage: Contiene un array de bytes sin interpretar. Se utiliza para hacer coincidir el cuerpo con un formato de mensaje existente (legacy).
  - StreamMessage: El cuerpo es un flujo de tipos primitivos Java, cuya lectura y escritura se realiza de modo secuencial.
  - TextMessage: Un mensaje cuya carga es un String. Se suele utilizar para enviar texto simple y datos XML o JSON.
  - ObjectMessage: La carga es un objeto Java Serializable. Normalmente se utiliza para trabajar con objetos Java complejos.

© JMA 2020. All rights reserved

ActiveMQ: Spring Boot

- Para añadir al proyecto ActiveMQ 5:
  - Messaging > Spring for Apache ActiveMQ 5
- Configurar:

```
#spring.activemq.broker-url=tcp://localhost:61616
#spring.activemq.user=admin
#spring.activemq.password=admin
spring.activemq.in-memory=false
```

- Para añadir al proyecto ActiveMQ Artemis:
  - Messaging > Spring for Apache ActiveMQ Artemis
- Configurar:

```
#spring.artemis.mode=native
spring.artemis.host=localhost
spring.artemis.port=61616
#spring.artemis.user=admin
#spring.artemis.password=admin
```

© JMA 2020. All rights reserved

Template y Anotaciones

- Spring proporciona un marco de integración de JMS que simplifica el uso del API de JMS de la misma manera que lo hace para JDBC.
- JMS se puede dividir aproximadamente en dos áreas de funcionalidad, a saber, la producción y el consumo de mensajes.
- La clase JmsTemplate se utiliza para la producción de mensajes y la recepción de mensajes sincrónicos.
- Para una recepción asincrónica similar al estilo de bean controlado por mensajes de Jakarta EE, Spring proporciona una serie de contenedores de escucha de mensajes que puede usar para crear POJO controlados por mensajes (MDP). Spring también proporciona una forma declarativa con anotaciones de crear oyentes de mensajes.

© JMA 2020. All rights reserved

Queues: enviar y recibir

- Enviar un mensaje:

```
@Autowired
JmsTemplate jms;

@GetMapping(path = "/saludo/{nombre}")
public String saluda(@PathVariable String nombre) {
        String msg = "Hola " + nombre;
        jms.convertAndSend("saludos", new MessageDTO(msg, origen));
        return "SEND COLA: " + msg;
}
```

- Recibir mensajes:

```
@JmsListener(destination = "saludos")
public void listenQueue(MessageDTO in) {
        Store.addQueue(new Message(in));
        log.warn("MENSAJE RECIBIDO: " + in);
}
```

© JMA 2020. All rights reserved

Topics: enviar y recibir

- Se puede establecer la Publicación/Subscripción como mecanismo por defecto en application.properties:

```
spring.jms.pub-sub-domain=true
```

- En caso de convivir los dos sistemas hay que crear uno o varios contenedores personalizados para asignarlos a los listener:

```
@Bean
JmsListenerContainerFactory<?> myPubSubFactory(ConnectionFactory connectionFactory,
        DefaultJmsListenerContainerFactoryConfigurer configurer) {
    DefaultJmsListenerContainerFactory factory = new DefaultJmsListenerContainerFactory();
    configurer.configure(factory, connectionFactory);
    factory.setPubSubDomain(true);
    return factory;
}
```

© JMA 2020. All rights reserved

Topics: enviar y recibir

- Enviar un mensaje:

```
@Autowired
JmsTemplate jms;
@GetMapping(path = "/despedida/{nombre}")
public String despedida(@PathVariable String nombre) {
        String msg = "Adios " + nombre;
        jms.setPubSubDomain(true);  // Si por defecto spring.jms.pub-sub-domain=false
        jms.convertAndSend("despedidas", new MessageDTO(msg, origen));
        return "SEND TEMA: " + msg;
}
```

- Recibir mensajes:

```
@JmsListener(destination = "despedidas", containerFactory = "myPubSubFactory")
public void listen(MessageDTO in) {
        Store.addTopic(new Message(in));
        log.warn("TEMA RECIBIDO: " + in);
}
```

© JMA 2020. All rights reserved

Formato del Mensaje

- Los objetos ObjectMessage dependen de la serialización de Java de la carga útil del objeto marshal/unmarshal. Este proceso generalmente se considera inseguro ya que una carga útil maliciosa puede explotar el sistema host. Es por eso que a partir de las versiones 5.12.2 y 5.13.0 , ActiveMQ obliga a incluir explícitamente en la lista blanca de los paquetes que se pueden intercambiar.

  -Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*

- Para facilitar el envío de objetos del modelo de dominio, JmsTemplate tiene varios métodos de envío que toman un objeto Java como argumento para el contenido de datos de un mensaje. Los métodos sobrecargados convertAndSend() y receiveAndConvert() delegan el proceso de conversión a una instancia de la interfaz MessageConverter. Esta interfaz define un contrato simple para convertir entre objetos Java y mensajes JMS. La implementación predeterminada (SimpleMessageConverter) admite la conversión entre String a TextMessage, byte[] a BytesMessage y java.util.Map a MapMessage.

© JMA 2020. All rights reserved

Formato del Mensaje

- Los objetos ObjectMessage dependen de la serialización de Java de la carga útil del objeto marshal/unmarshal. Este proceso generalmente se considera inseguro ya que una carga útil maliciosa puede explotar el sistema host. Es por eso que a partir de las versiones 5.12.2 y 5.13.0 , ActiveMQ obliga a incluir explícitamente en la lista blanca de los paquetes que se pueden intercambiar.

  -Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*
- Para facilitar el envío de objetos del modelo de dominio, JmsTemplate tiene varios métodos de envío que toman un objeto Java como argumento para el contenido de datos de un mensaje. Los métodos sobrecargados convertAndSend() y receiveAndConvert() delegan el proceso de conversión a una instancia de la interfaz MessageConverter. Esta interfaz define un contrato simple para convertir entre objetos Java y mensajes JMS. La implementación predeterminada (SimpleMessageConverter) admite la conversión entre String a TextMessage, byte[] a BytesMessage y java.util.Map a MapMessage.

© JMA 2020. All rights reserved

Formato del Mensaje

- Para establecer un conversor predeterminado:

```
@Bean // Serialize message content to json using TextMessage
MessageConverter jacksonJmsMessageConverter() {
        MappingJackson2MessageConverter converter = new MappingJackson2MessageConverter();
        converter.setTargetType(MessageType.TEXT);
        converter.setTypeIdPropertyName("_type");
        return converter;
}
```

- Para acomodar la configuración de las propiedades, los encabezados y el cuerpo de un mensaje que no se pueden encapsular genéricamente dentro del convertidor, la interfaz MessagePostProcessor brinda acceso al mensaje después de que se haya convertido pero antes de que se envíe.

```
public void sendWithConversion(Map map) {
   jmsTemplate.convertAndSend("testQueue", map, new MessagePostProcessor() {
      public Message postProcessMessage(Message message) throws JMSException {
         message.setIntProperty("AccountID", 1234);
         message.setJMSCorrelationID("123-00001");
         return message;
      }
   });
}
```

© JMA 2020. All rights reserved

WEBSOCKET Y STOMP

© JMA 2020. All rights reserved

WebSocket

- El protocolo WebSocket, RFC 6455, proporciona un canal de comunicación bidireccional y full-dúplex, entre el cliente y el servidor, a través de una única conexión TCP. Es un protocolo TCP diferente de HTTP, pero está diseñado para funcionar sobre HTTP, utilizando los puertos 80 y 443 y permitiendo la reutilización de las reglas de firewall existentes.
- La API de WebSocket ha sido normalizada por el W3C, mientras que el protocolo WebSocket fue normalizado por la IETF como el RFC 6455.
- WebSockets es una tecnología avanzada que hace posible abrir una sesión de comunicación interactiva entre el navegador del usuario y un servidor. Con la API de navegador, se puede enviar mensajes a un servidor y recibir respuestas controladas por eventos sin tener que consultar al servidor para una respuesta: una vez establecida la conexión por parte del cliente, el servidor puede hacer envíos al cliente sin esperar a una petición.

© JMA 2020. All rights reserved

HTTP frente a WebSocket

- Aunque WebSocket está diseñado para ser compatible con HTTP y comienza con una solicitud HTTP, es importante comprender que los dos protocolos conducen a arquitecturas y modelos de programación de aplicaciones muy diferentes.
- En HTTP y REST, una aplicación se modela como muchas URL. Para interactuar con la aplicación, los clientes acceden a esas URL, estilo solicitud-respuesta. Los servidores enrutan las solicitudes al controlador apropiado en función de la URL, el método HTTP y los encabezados.
- Por el contrario, en WebSockets, generalmente solo hay una URL para la conexión inicial. Posteriormente, todos los mensajes de la aplicación fluyen en esa misma conexión TCP. Esto apunta a una arquitectura de mensajería asincrónica, impulsada por eventos, completamente diferente.
- WebSocket también es un protocolo de transporte de bajo nivel que, a diferencia de HTTP, no prescribe ninguna semántica para el contenido de los mensajes. Eso significa que no hay forma de enrutar o procesar un mensaje a menos que el cliente y el servidor acuerden la semántica del mensaje.
- Los clientes y servidores de WebSocket pueden negociar el uso de un protocolo de mensajería de nivel superior (por ejemplo, STOMP), a través del encabezado Sec-WebSocket-Protocol en la solicitud de protocolo de enlace HTTP. En ausencia de eso, necesitan llegar a sus propias convenciones.

© JMA 2020. All rights reserved

WebSocket

- El protocolo WebSocket define un prefijo ws:// y wss:// para indicar una conexión WebSocket y Websocket Secure, respectivamente. Una interacción de WebSocket comienza con una solicitud HTTP que usa el encabezado HTTP Upgrade para actualizar o cambiar al protocolo WebSocket:

```
GET /chat HTTP/1.1
Host: localhost:8080
Connection: Upgrade
Upgrade: websocket
Sec-WebSocket-Version: 13
Sec-WebSocket-Key: Uc9l9TMkWGbHFD2qnFHltg==
Origin: http://localhost:8080
```

- En lugar del código de estado 200 habitual, un servidor compatible con WebSocket devuelve:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: upgrade
Sec-WebSocket-Accept: DoDFpVvCTJDVTPPfR9lEFDIt5A0=
Sec-WebSocket-Extensions: permessage-deflate;client
```

© JMA 2020. All rights reserved

Casos de uso

- WebSockets puede hacer que una página web sea dinámica e interactiva, es especialmente bueno para servicios que requieren intercambio de información continua, por ejemplo juegos en línea, sistemas de negocios en tiempo real, entre otros. Sin embargo, en muchos casos, una combinación de transmisión Ajax y HTTP o un sondeo largo puede proporcionar una solución simple y efectiva.
- La latencia por sí sola no es un factor decisivo. Si el volumen de mensajes es relativamente bajo (por ejemplo, al monitorizar fallos en la red), la transmisión o el sondeo HTTP pueden brindar una solución efectiva. Es la combinación de baja latencia, alta frecuencia y alto volumen lo que hace el mejor caso para el uso de WebSocket.
- Hay que tener en cuenta también que en Internet, los proxies restrictivos que están fuera de nuestro control pueden impedir las interacciones de WebSocket, ya sea porque no están configurados para transmitir el encabezado Upgrade o porque cierran conexiones de larga duración que parecen inactivas. Esto significa que el uso de WebSocket para aplicaciones internas detras del firewall es una decisión más sencilla que para aplicaciones públicas.
- El mantener conexiones abiertas es costoso en recursos del servidor (cuando hay muchos usuarios), no son fáciles de testear y hay que hacerlas tolerante a fallos manualmente.

© JMA 2020. All rights reserved

WebSocket en Spring

- Spring Framework proporciona una API de WebSocket que se puede usar para escribir aplicaciones del lado del cliente y del servidor que manejen mensajes de WebSocket.
- Añadir al proyecto:
  - Messaging > WebSocket
- Crear un servidor WebSocket es tan simple como implementar WebSocketHandler, probablemente extendiendo TextWebSocketHandler o BinaryWebSocketHandler, y registrarlo en la configuración al habilitar los WebSockets.

```
@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {
    @Autowired SocketTextHandler cliente;
     @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(cliente, "/chat");
    }
}
```

© JMA 2020. All rights reserved

TextWebSocketHandler

```
@Component
public class SocketTextHandler extends TextWebSocketHandler {
	@Autowired SocketService clientes;
	@Override
	public void afterConnectionEstablished(WebSocketSession session) throws Exception {
		if (session.getUri().getQuery() == null) {
			session.close(new CloseStatus(1006, "Falta el nombre de usuario"));
		} else {
			clientes.add(session.getUri().getQuery(), session);
			session.sendMessage(new TextMessage("Servidor: Te has conectado como " + session.getUri().getQuery()));
		}
		super.afterConnectionEstablished(session);
	}
	@Override
	public void afterConnectionClosed(WebSocketSession session, CloseStatus status) throws Exception {
		clientes.remove(session);
		super.afterConnectionClosed(session, status);
	}
	@Override
	public void handleTextMessage(WebSocketSession session, TextMessage message)  throws InterruptedException, IOException {
		clientes.broadcast(clientes.getName(session) + ": " + message.getPayload());
	}
}
```

© JMA 2020. All rights reserved

Cliente JavaScript

- Para abrir una conexión websocket, se crea un new WebSocket usando el protocolo ws o wss (seguro) en la url:

  let socket = new WebSocket(`**ws**://${location.host}/chat`);
- Para obtener el estado (state) de la conexión, tenemos la propiedad socket.readyState con valores:
  - 0: “CONNECTING”, la conexión aún no fue establecida
  - 1: “OPEN”, la conexión esta disponible para la comunicación
  - 2: “CLOSING”, la conexión se está cerrando
  - 3: “CLOSED”, la conexión está cerrada
- Una vez que el socket es creado, debemos escuchar los eventos que ocurren en él. Hay en total 4 eventos:
  - open: conexión establecida,
  - message: datos recibidos,
  - error: error en websocket,
  - close: conexión cerrada.

© JMA 2020. All rights reserved

Cliente JavaScript

- Una vez abierta la conexión se utilizan los métodos:
  - socket.send(data),
  - socket.close([code], [reason]).
- La comunicación WebSocket consiste de frames (cuadros) de fragmentos de datos, que pueden ser enviados de ambos lados y pueden ser de varias clases:
  - text frames: contiene datos de texto que las partes se mandan entre sí.
  - binary data frames: contiene datos binarios que las partes se mandan entre sí.
  - ping/pong frames: son usados para testear la conexión; enviados desde el servidor, el navegador responde automáticamente.
  - También existe “connection close frame”, y algunos otros frames de servicio.
- En el navegador, el método WebSocket .send(data) puede enviar tanto datos de texto como binarios.

```
socket.send(message)
```

© JMA 2020. All rights reserved

Cliente JavaScript

- Cuando recibimos datos, el texto siempre viene como string. Y para datos binarios, podemos elegir entre los formatos Blob ("blob") y ArrayBuffer ("arraybuffer"). Blob es un objeto binario de alto nivel que se integra directamente con <a>, <img> y otras etiquetas, pero para procesamiento binario y acceder a bytes individuales, podemos cambiarlo a "arraybuffer":

```
socket.binaryType = "arraybuffer";
socket.onmessage = (event) => {
```

- Normalmente, cuando una parte quiere cerrar la conexión (servidor o navegador, ambos tienen el mismo derecho), envía un “frame de cierre de conexión” con un código numérico y un texto con el motivo.

```
socket.close([code], [reason]);
```

- Los códigos más comunes:
  - 1000: cierre normal. Es el predeterminado (usado si no se proporciona code),
  - 1006: no hay forma de establecerlo manualmente, indica que la conexión se perdió (sin frame de cierre).
  - 1001: una parte se va, por ejemplo el server se está apagando, o el navegador deja la página,
  - 1009: el mensaje es demasiado grande para procesar,
  - 1011: error inesperado en el servidor,
  - La lista completa puede encontrarse en RFC6455, §7.4.1.

© JMA 2020. All rights reserved

Cliente JavaScript

```
<script>
let socket = null;
const connect = () => {
	socket = new WebSocket(`ws://${location.host}/chat`);
	socket.onopen = () => show("[open] Conexión establecida");
	socket.onmessage = event => show(`[message] ${event.data}`);
	socket.onclose = event => {
		show(`[close] Conexión cerrada ${event.wasClean ? 'limpiamente' : 'con problemas'}, código: ${event.code} ${event.reason}`);
	}
	socket.onerror = event => show(`[error]: ${JSON.stringify(error)}`);
}
const send = message => {
	if(socket && socket.readyState === 1)
		socket.send(message)
}
const disconnect = () => {
	if(socket && socket.readyState === 1)
		socket.close()
}
</script>
```

© JMA 2020. All rights reserved

Cliente Java

```
import org.springframework.web.socket.TextMessage;
import org.springframework.web.socket.WebSocketSession;
import org.springframework.web.socket.client.WebSocketClient;
import org.springframework.web.socket.client.standard.StandardWebSocketClient;
import org.springframework.web.socket.handler.TextWebSocketHandler;

	private WebSocketSession client;

		if (client == null || !client.isOpen()) {
			WebSocketClient ws = new StandardWebSocketClient();
			client = ws.execute(new TextWebSocketHandler(), WEBSOCKET_SERVER_URL).get();
		}
		if(client.isOpen()) {
			client.sendMessage(new TextMessage("[" + message.getName() + "]: " + message.getText()));
		} else {
			log.error("Sesion cerrada");
		}
```

© JMA 2020. All rights reserved

SockJS

- En Internet (pública), los proxies restrictivos fuera de nuestro control pueden impedir las interacciones de WebSocket, ya sea porque no están configurados para transmitir el encabezado Upgrade o porque cierran conexiones de larga duración que parecen estar inactivas.
- La solución a este problema es la emulación de WebSocket, es decir, intentar usar WebSocket primero y luego recurrir a técnicas basadas en HTTP que emulan una interacción de WebSocket y exponen la misma API de nivel de aplicación.
- En la pila de servlet, Spring Framework proporciona soporte tanto para el servidor (como para el cliente) para el protocolo SockJS.
- El objetivo de SockJS es permitir que las aplicaciones usen una API de WebSocket pero, en tiempo de ejecución, recurran a alternativas que no sean de WebSocket cuando no este soportado, sin necesidad de cambiar el código de la aplicación.
- SockJS consta de:
  - El protocolo SockJS definido entre el cliente en el navegador y sus contrapartes del lado del servidor.
  - El cliente JavaScript SockJS: una biblioteca cliente para usar en navegadores.
  - Implementaciones del servidor SockJS (hay una incluida en el módulo spring-websocket de Spring Framework).
  - Un cliente SockJS Java (también incluida en el módulo spring-websocket desde la versión 4.1).

© JMA 2020. All rights reserved

SockJS

- El cliente SockJS comienza enviando GET /info para obtener información básica del servidor. Después de eso, debe decidir qué transporte utilizar. Si es posible, se utiliza WebSocket. Si no, en la mayoría de los navegadores, hay al menos una opción de transmisión HTTP (HTTP Streaming, Server Sent Events). De lo contrario, se utiliza el sondeo HTTP largo (HTTP Long Polling).
- Se puede activar al registrar el WebSocketHandler:

```
registry.addHandler(cliente, "/chat").setAllowedOrigins("*");
```

- Para acceder desde el navegador:

```
<script src="https://cdn.jsdelivr.net/npm/sockjs-client@1/dist/sockjs.min.js"></script>
<script>
var socket = new SockJS('/chat');
socket.onopen = () => …
socket.onmessage = event => …
socket.onclose = event => …
socket.onerror = event => …
socket.send(message)
socket.close()
```

© JMA 2020. All rights reserved

Orígenes permitidos

- A partir de Spring Framework 4.1.5, el comportamiento predeterminado de WebSocket y SockJS es aceptar solo solicitudes del mismo origen cuando el clientes es un navegador. También es posible permitir todos o una lista específica de orígenes. Nada impide que otros tipos de clientes modifiquen el valor del encabezado Origin.
- Los tres comportamientos posibles son:
  - Permitir solo solicitudes del mismo origen (predeterminado): en este modo, cuando SockJS está habilitado, el encabezado de respuesta HTTP de Iframe X-Frame-Options se establece en SAMEORIGIN y el transporte JSONP está deshabilitado, ya que no permite verificar el origen de una solicitud.
  - Permitir una lista específica de orígenes: cada origen permitido debe comenzar con http:// o https://. En este modo, cuando SockJS está habilitado, el transporte de IFrame está deshabilitado.
  - Permitir todos los orígenes: para habilitar este modo, se debe proporcionar el * como valor de origen permitido. En este modo, todos los transportes están disponibles.
- Se puede configurar los orígenes permitidos al registrar el WebSocketHandler:

```
registry.addHandler(cliente, "/chat").setAllowedOrigins("*");
```

© JMA 2020. All rights reserved

STOMP sobre WebSocket

- El protocolo WebSocket define dos tipos de mensajes (texto y binario), pero su contenido no está definido. El protocolo define un mecanismo para que el cliente y el servidor negocien un subprotocolo (es decir, un protocolo de mensajería de nivel superior) para usar sobre WebSocket para definir qué tipo de mensajes puede enviar cada uno, cuál es el formato, el contenido de cada mensaje, y así sucesivamente. El uso de un subprotocolo es opcional pero, de cualquier manera, el cliente y el servidor deben acordar algún protocolo que defina el contenido del mensaje.
- STOMP (Protocolo de mensajería orientado a texto simple) define un formato de conexión interoperable para que cualquiera de los clientes STOMP disponibles pueda comunicarse con cualquier intermediario de mensajes STOMP para proporcionar una interoperabilidad de mensajería fácil y generalizada entre lenguajes y plataformas. Se creó originalmente para que lenguajes de secuencias de comandos (como Ruby, Python y Perl) se conectasen a intermediarios de mensajes empresariales. Está diseñado para abordar un subconjunto mínimo de patrones de mensajería de uso común y se puede usar sobre cualquier protocolo de red de transmisión bidireccional confiable, como TCP y WebSocket. Aunque es un protocolo orientado a texto, las cargas de mensajes pueden ser de texto o binarias.

© JMA 2020. All rights reserved

Protocolo orientado a texto

- Los clientes pueden usar los comandos SEND o SUBSCRIBE para enviar o suscribirse a mensajes, junto con un encabezado *destination* que describe de qué se trata el mensaje y quién debe recibirlo. Esto habilita un mecanismo simple de publicación y suscripción que puede usar para enviar mensajes a través del intermediario a otros clientes conectados o para enviar mensajes al servidor para solicitar que se realice algún trabajo.

```
CONNECT
login:guest
passcode:guest

    CONNECTED
    session:1092296064

SEND
destination:/queue/test

Hello from TCP!

SUBSCRIBE
id:sub-1
destination:/topic/price.stock.*
```

© JMA 2020. All rights reserved

STOMP con Spring

- Cuando utiliza la compatibilidad con STOMP de Spring, la aplicación Spring WebSocket actúa como intermediario de STOMP para los clientes. Los mensajes se enrutan a los métodos de manejo de mensajes de un @Controller o a un intermediario en memoria simple que realiza un seguimiento de las suscripciones y transmite mensajes a los usuarios suscritos.
- También se puede configurar Spring para trabajar con un agente STOMP dedicado (como RabbitMQ, ActiveMQ y otros) para la transmisión real de mensajes. En ese caso, Spring mantiene conexiones TCP con el intermediario, le transmite mensajes y pasa mensajes desde él a los clientes WebSocket conectados. Por lo tanto, las aplicaciones web de Spring pueden confiar en la seguridad unificada basada en HTTP, la validación común y un modelo de programación familiar para el manejo de mensajes.
- El uso de STOMP como subprotocolo permite que Spring Framework y Spring Security proporcionen un modelo de programación más rico en comparación con el uso de WebSockets sin procesar. Los beneficios mas destacables son:
  - No es necesario inventar un protocolo de mensajería personalizado y un formato de mensaje.
  - Los clientes STOMP, incluido un cliente Java en Spring Framework, ya están disponibles.
  - Se pueden usar intermediarios de mensajes (como RabbitMQ o ActiveMQ y otros) para administrar suscripciones y difundir mensajes.
  - La lógica de la aplicación se puede organizar en cualquier cantidad de instancias @Controller y los mensajes se pueden enrutar a ellas en función del encabezado de destino de STOMP en lugar de manejar los mensajes WebSocket sin procesar con un único WebSocketHandler para una conexión determinada.
  - Se puede usar Spring Security para proteger los mensajes en función de los destinos y tipos de mensajes de STOMP.

© JMA 2020. All rights reserved

Configurar el servidor

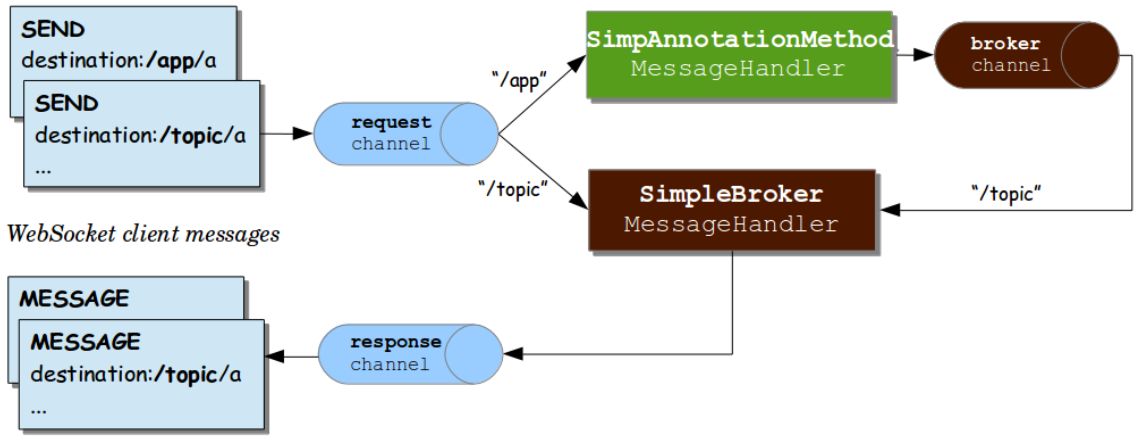
- La compatibilidad con STOMP sobre WebSocket está disponible en los módulos spring-messaging y spring-websocket.
- Para configurar el servidor WebSocket subyacente:

```
@Configuration
@EnableWebSocketMessageBroker
public class StompWebSocketConfig implements WebSocketMessageBrokerConfigurer {
  @Override public void registerStompEndpoints(StompEndpointRegistry registry) {
    registry.addEndpoint("/chat").withSockJS();
  }
  @Override public void configureMessageBroker(MessageBrokerRegistry config) {
    config.setApplicationDestinationPrefixes("/app");
    config.enableSimpleBroker("/topic", "/queue");
  }
}
```

- Donde /chat es la URL HTTP para el punto final al que un cliente WebSocket (o SockJS) necesita conectarse para el protocolo de enlace WebSocket. Los mensajes STOMP cuyo encabezado de destino comienza con /app se enrutan a los métodos @MessageMapping de los @Controller. Se habilita el agente de mensajes integrado para suscripciones y transmisiones, para que enrute los mensajes cuyo encabezado de destino comience con /topic o /queue a los métodos @SendTo de los @Controller.

© JMA 2020. All rights reserved

Flujo de mensajes



© JMA 2020. All rights reserved

Controladores anotados

- Las aplicaciones pueden usar clases anotadas con @Controller para manejar mensajes de clientes. Dichas clases pueden declarar métodos:
  - @MessageMapping: para anotar métodos que enrutan mensajes en función de su destino.
  - @SubscribeMapping: limita la asignación a los mensajes de suscripción únicamente.
  - @MessageExceptionHandler: para manejar las excepciones de los métodos @MessageMapping.

```
@Controller
@RequiredArgsConstructor
public class ChatController {
   private final StompService socketService;
   @MessageMapping("/chat")
   @SendTo("/topic/messages")
   public MessageDto sendMessage(MessageDto message, @Header("simpSessionId") String sessionId) {
      socketService.saveSession(sessionId, message.getName());
      return message;
   }
}
```

© JMA 2020. All rights reserved

SimpMessagingTemplate

- Cualquier componente de la aplicación puede enviar mensajes al brokerChannel. La forma más sencilla de hacerlo es inyectar un SimpMessagingTemplate y usarlo para enviar mensajes.

```
@Controller
public class GreetingController {
   @Autowired
   private SimpMessagingTemplate template;
   @PostMapping(path="/saluda")
   public void send(String name) {
      template.convertAndSend("/topic/messages", "Hola " + name);
   }
}
```

© JMA 2020. All rights reserved

Simple Broker

- El intermediario de mensajes simple (Simple Broker) incorporado maneja las solicitudes de suscripción de los clientes, las almacena en la memoria y transmite mensajes a los clientes conectados que tienen destinos coincidentes. El bróker admite destinos similares a las rutas.

```
@Configuration
@EnableWebSocketMessageBroker
public class StompWebSocketConfig implements WebSocketMessageBrokerConfigurer {
   @Lazy @Autowired
   private TaskScheduler messageBrokerTaskScheduler;
   @Override
   public void configureMessageBroker(MessageBrokerRegistry registry) {
      registry.enableSimpleBroker("/queue/", "/topic/")
            .setHeartbeatValue(new long[] {10000, 20000})
            .setTaskScheduler(this.messageBrokerTaskScheduler);
      :
   }
}
```

© JMA 2020. All rights reserved

External Broker

- El intermediario simple es excelente para comenzar, pero admite solo un subconjunto de comandos STOMP (no admite confirmaciones, recibidos y algunas otras funciones), se basa en un ciclo simple de envío de mensajes y no es adecuado para la agrupación.
- Como alternativa, se pueden actualizar las aplicaciones para utilizar un intermediario de mensajes externo (como RabbitMQ, ActiveMQ y otros) con todas las funciones. El intermediario debe ejecútarse con la compatibilidad con STOMP habilitada.
- Para habilitar el reenvío al intermediario de mensajes externo en la configuración:

```
@Configuration
@EnableWebSocketMessageBroker
public class StompWebSocketConfig implements WebSocketMessageBrokerConfigurer {
   :
   @Override public void configureMessageBroker(MessageBrokerRegistry config) {
      config.setApplicationDestinationPrefixes("/app");
      config.enableStompBrokerRelay("/topic", "/queue");
   }
}
```

© JMA 2020. All rights reserved

Cliente JavaScript

```
<script>
    let stompClient = null;
    let unsubscribe = null;
    const connect = () => {
        stompClient = Stomp.client(`ws://${location.host}/chat`);
        unsubscribe = stompClient.connect({}, frame => {
            stompClient.subscribe('/topic/messages', function (response) {
                let messaje = JSON.parse(response.body)
                show(`${messaje.name}: ${messaje.text}`)
            });
        }, error => show(`[error]: ${JSON.stringify(error)}`));
    }
    const send = message => {
        stompClient.send("/example/chat", {}, JSON.stringify({ 'name': username, text: input.value }));
    }
    const disconnect = () => {
        if (unsubscribe) unsubscribe();
        stompClient.disconnect(() => show(`[close] Disconnect`));
    }
</script>
```

© JMA 2020. All rights reserved

WebJars

- Dependencias (en Maven):

```
<dependency>
    <groupId>org.webjars</groupId>
    <artifactId>webjars-locator-core</artifactId>
</dependency>
<dependency>
    <groupId>org.webjars</groupId>
    <artifactId>sockjs-client</artifactId>
    <version>1.0.2</version>
</dependency>
<dependency>
    <groupId>org.webjars</groupId>
    <artifactId>stomp-websocket</artifactId>
    <version>2.3.4</version>
</dependency>
```

- Referenciado (en HTML):

```
<script src="/webjars/sockjs-client/sockjs.min.js"></script>
<script src="/webjars/stomp-websocket/stomp.min.js"></script>
```

© JMA 2020. All rights reserved

Cliente Java

```
WebSocketClient client = new StandardWebSocketClient();
stompClient = new WebSocketStompClient(client);
stompClient.setMessageConverter(new MappingJackson2MessageConverter());
stompClient.connectAsync(STOMP_SERVER_URL, new StompSessionHandlerAdapter() {
   private static final Logger log = LoggerFactory.getLogger("StompSessionHandlerAdapter");
   @Override
   public void afterConnected(StompSession session, StompHeaders connectedHeaders) {
      session.subscribe("/topic/messages", this);
      session.send("/example/chat", new MessageDto("[REMOTE]", "Me he conectado!!!"));
   }
   @Override
   public void handleException(StompSession session, StompCommand command, StompHeaders headers, byte[] payload, Throwable exception) {
      log.error("Got an exception", exception);
   }
   @Override public Type getPayloadType(StompHeaders headers) { return MessageDto.class; }
   @Override
   public void handleFrame(StompHeaders headers, Object payload) {
       MessageDto msg = (MessageDto) payload;
       try {
           clientes.broadcast("[STOMP " + msg.getName() + "]: " + msg.getText());
       } catch (IOException e) { e.printStackTrace(); }
   }
});
```

© JMA 2020. All rights reserved